Vers une Approche de Remédiation Automatique de Code Source pour la Sécurité et la Confidentialité de Tierces Applications Mobiles



Mémoire de fin d'étude

Master *Sciences et Technologies*,
Mention *Informatique*,
Parcours Génie Logiciel

Auteur
Louis Parent

Superviseurs
Abdelhak-Djamel Seriai

Lieu de stage
LIRMM, Université de Montpellier

version 1 soutenu publiquement le 23 juin 2023

Résumé

L'informatique et les données sont au coeur de nos vies. La démocratisation des appareils mobiles a d'autant plus banalisé le stockage des données privées. La protection des données et des applications qui les contiennent est un enjeu majeur de la sécurité informatique. La lutte face à la cybercriminilatié est un jeu constant du chat et de la souris où les attaquants cherchent sans cesse à contourner les dernières sécurtiés mises en place. C'est dans ce contexte que le blindage a vu le jour en prenant un angle de défense disuasif. Il a pour but de complexifier la tâche de rétro-ingénierie afin de rendre plus ardue le détournement des fonctionnalités de l'application ou du vol de données. Durant ce stage, nous avons étudié l'approche du blindage. Nous avons proposé un jeu de vulnérabilités qui doivent être prises en compte par le blindage. Nous avons aussi proposé différentes remédiations à ces vulnérabilités afin d'éviter de perturber le fonctionement normal de l'application. Enfin, nous avons vérifié notre démarche en la mettant en oeuvre à travers un outil de blindage automatisé.

Abstract

Nowadays, computer science and data are everywhere in out lives. The usage of smartphone has skyrocketed, trivialize the storage of private data. The protection of this data and the associated applications is one of the most important challenges in cybersecurity. The fight against these threats is a hide and seek game between attacker and defender. Attackers are always looking for a breach in the new protection being set up. It's in this context which shielding was created. It uses the principle of dissuasive protection by complexing reverse engineering to avoid data leakage and application tampering. During this internship, we study the shielding approach. We proposed a set of vulnerabilities against which the shielding must take over. We also proposed a set of remediation to these vulnerabilities to avoid disturbing the application behaviour. We end by testing our methodology with an automated tool for shielding.

Remerciements

Je tiens à remercier *M. Abdelhak-Djamel Seriai* pour l'opportunité qu'il m'a offerte et la confiance qu'il m'a accordé en me permettant de réaliser ce stage de recherche. Je le remercie aussi pour son introduction et son assistance dans la démarche de recherche qu'il m'a été nécessaire d'appréhender. Enfin, je le remercie pour son aide dans la rédaction de ce rapport.

Je tiens aussi à remercier Isaac LLuís pour avoir pris le temps de relire ce rapport.

Glossaire

réseau de neurones artificiels récurrents Un réseau de neurones artificiels récurrents (*Recurrent Neural Network* abrégé **RNN**) est un réseau de neurones artificiels autorisant les cycles dans les connections entre les neurones. 13

Android *Android* est un système d'exploitation pour appareils mobiles basé sur le noyau *Linux* développé par un consortium d'entreprises du secteur mobile (*Open Handset Alliance*). 1, 5, 9, 14–27, 29, 30, 32, 37, 39

C/C++ Le *C* est un langage de programmation impératif de bas niveau développé par *Bell Labs*. Le *C++* est un langage inspiré et compatible avec le *C* rajoutant de multiples paradigmes notamment l'orienté-objet. 16, 21, 22, 37

ECMAScript *ECMAScript*, ou *ECMA-262*, est une norme mise en place par *ECMA* afin de standardiser l'implémentation de *JavaScript*. 5

GSM Le *GSM*, *Global System for Mobile Communications* en anglais, est le standard pour le réseau mobile de seconde génération (*2G*). Malgrès l'arrêt de ce réseau suite aux évolutions *3G*, *4G*,*5G*, le *GSM* reste un acronyme utilisé pour parlé du réseau mobile quelque soit sa génération. 20, 24, 27

JavaScript Le *JavaScript* est un langage orienté protype dévelopé à l'origine par *Netscape* pour leur navigateur. Il s'est imposé comme un standard pour les scripts web, et est maintenant normalisé par *ECMA* sous le nom *ECMAScript*. 5

Java Le *Java* est un langage portable orienté-objet originellement développé par *Sun Microsystems* puis racheté par *Oracle*. 16, 21, 22, 26–28, 30, 32

MacOS *MacOS* est un OS pour oridinateur développé par *Apple*. 5

Rust *Rust* est un langage de programmation développé par *Mozilla*. 30

Smali Le *Smali* est un langage comparable à l'assembleur mais ciblé pour la machine virtuelle *Android*. Il intégre la logique objet de *Java*. 30, 32, 34

TCP/IP Le *TCP/IP* est un ensemble de protocoles utilisé pour la communication internet. Son nom provient des protocoles *TCP* et *IP* qui en sont les protocoles fondateurs. 4

Unix *Unix* est un standard pour système d'exploitation dérivé du système du même nom développé par *Bell Labs*. 5

VPN Un *VPN*, *Virtual Private Network* en anglais, est un système de communication sécurisée permettant à un ordinateur de se connecter à un réseau distant à travers internet. 24, 28

Windows *Windows* est un OS pour ordinateur et serveur développé par *Microsoft*. 5

iOS *iOS* est un OS dédié aux appareils mobiles développé par *Apple*. 5, 37, 39

multidex Le *multidex* est une extension au format du fichier *classes.dex* en permettant d'en inclure plusieurs au sein d'un même APK afin de palier à la limite de méthode par fichier *.dex*. 37, 39

accrochage L'accrochage (*Hooking* en anglais) est une pratique permettant de modifier le comportement d'un programme en interceptant un évènement ou un appel de fonction. 15, 16, 20, 21, 37

activité Une activité (sous-classe de la classe *Activity*) est le composant de base d'une application *Android* permettant de décrire un écran avec lequel l'utilisateur peut intéragir. 17, 29

analyse formelle de concepts L'analyse formelle de concept (*Formal Concept Analysis* abrégé **FCA**) est une application de la théorie des treillis afin d'étudier les concepts décrits formellement. 12

APK L'APK est le format des paquets applicatifs *Android*. Ils contiennent l'ensemble des fichiers requis pour installer et utiliser une application. On peut citer en particulier le fichier manifeste android, le dossier de ressources *res* ou encore le fichier de code binaire *classes.dex*. 10, 29, 30, 32, 36, 39

apprentissage automatique L'apprentissage automatique (*Machine Learning* en anglais) est un domaine de l'intelligence artificielle permettant de développer un modèle à partir de données afin de simuler un apprentissage. 9, 11, 13

apprentissage profond L'apprentissage profond (*Deep Learning* en anglais) est un type d'apprentissage automatique utilisant un réseau de neurones artificiels à plusieurs couches comme modèle. 13

arbre de la syntaxe abstraite Un arbre de la syntaxe abstraite (*Abstract Syntax Tree* abrégé **AST**) est une structure de donnée arborescente permettant de représenter un ensemble ordonné d'instructions où un noeud est une opération et les enfants sont ses opérandes. 12

auto-protection des applications en exécution L'auto-protection des applications en exécution (*Runtime Application Self-Protection* abrégé **RASP**) est une branche de la sécurité logicielle permettant de protéger un programme durant son exécution. 8

bibliothèque Une bibliothèque est un ensemble de fonctions, méthodes ou classes distribuées aux développeurs et prêtes à être réutilisées. 28, 30, 32, 34, 36

blindage Le blindage (*Shielding* en anglais) est un ensemble de techniques permettant de sécuriser le fonctionnement d'une application ainsi que son intégrité. 1, 7, 8, 10, 14, 16, 18, 19, 30–32, 36, 37, 39

cadriciel Un cadriciel (*Framework* en anglais) est un ensemble de composants logiciels réutilisables permettant de structurer les bases ainsi qu'une partie plus ou moins importante d'un logiciel. 5, 16, 21, 37

chiffrement en boite blanche Le chiffrement en boite blanche est le chiffrement du code d'un programme permettant de résister aux attaques dans un contexte de boite blanche, c'est à dire en connaissance totale du fonctionnement interne. 15

classe *Application* La classe *Application* est le composant racine d'une application *Android*. Une instance unique sera créée et accessible depuis le code de l'application lorsque celle-ci est démarrée. 32

contrat Un contrat est une spécification formelle d'une fonctionnalité d'un composant d'un programme . 12, 13

dorsal Le dorsal (*backend* en anglais) est la couche logicielle s'occupant de l'accès aux données et de la logique métier. On l'oppose généralement au frontal (*frontend* en anglais) qui est la couche de présentation affichée à l'utilisateur. 5, 7, 19

enregistreur de frappes Un enregistreur de frappes (*Keylogger* en anglais) est un programme ayant pour objectif d'enregistrer toutes les entrées claviers. 3, 15, 17, 26, 29

exécution symbolique L'exécution symbolique est un moyen d'analyser la façon de s'exécuter d'un programme en utilisant des valeurs symboliques pour les entrées à la place de valeurs réelles. 12

falsification La falsification (*Tampering* en anglais) est la modification intentionnelle et non authorisée d'un programme, de son comportement, ou de ses données dans un but généralement malveillant. 15, 16, 26, 29

fonction de base radiale Un réseau de fonctions de base radiale (*Radial Base Function* abrégé **BP**) est un réseau de neurones artificiels utilisant ces dites fonctions comme fonction d'activation. 11

grand modèle de langage Un grang modèle de langage (*Large Language Model* abrégé **LLM**) est algorithme d'intelligence artificielle qui utilise l'apprentissage profond associé à une quantité massive de données pour réaliser ses calculs. 13

hameçonnage L'hameçonnage (*Phishing* en anglais) est une technique de piratage informatique visant à faire croire à un individu qu'il est dans un contexte sécurisé afin qu'il révèle des informations confidentielles. 4

homme du milieu L'attaque de l'homme du milieu est une attaque informatique où l'attaquant se place comme intermédiaire dans la communication de deux entités sans qu'aucune des deux ne soit avertit. 20, 24

interface de programmation Une interface de programmation (*Application Programming Interface* abrégé **API**) est un type d'interface logicielle permettant de proposer à un autre programme un ensemble de fonctionnalités. 16

JAR Le JAR est le format de fichier de code compilé et compressé utilisé par *Java*. 30

kit de développement Un kit de développement (*Software Development Kit* abrégé **SDK**) est un ensemble d'outils logiciels permettant de faciliter le développement d'un logiciel en fournissant à la fois les outils de compilation/interprétation et des librairies. 28

lecteur d'écran Un lecteur d'écran (*Screenreader* en anglais) est un programme permettant de récupérer les informations d'un tier programme en analysant l'image envoyé à l'écran de l'utilisateur. 16, 17, 26, 29

liaison avec l'appareil La liaison avec l'appareil (*Binding* en anglais) est un technique permettant d'associer un programme à un identifiant physique, ne lui permettant ainsi de ne s'exécuter que sur l'appareil possédant cet identifiant. 16, 17, 26

logique temporelle linéaire La logique temporelle linéraire (*Linear Temporal Logic* abrégé **LTL**) est une logique modale qui étend la logique formelle avec des modalités se référent au temps. 12

maliciel Un maliciel (*Malware* en anglais) est un programme intentionnellement malveillant. 7, 19

manifeste android Le manifeste *Android* (*AndroidManifest.xml*) est un fichier contenu dans tout les APK permettant de définir une application avec ses propriétés et ses différents points d'entrées. 30, 32

modèle OSI Le modèle OSI est un modèle en couche permettant de décrire l'organisation des communications inter-système qui a été adoptée comme standard. 4

N-gramme Un N-gramme est une séquence continue de N éléments tirés d'un échantillon textuel ou vocal. 12

obfuscation L'obfuscation est un ensemble de techniques visant à complexifier un code dans le but de le rendre difficilement lisible et compréhensible. 14, 15, 29

programmation génétique La programmation génétique (*Genetic Programmation* abrégé **GP**) est une méthode de programmation automatique qui se base sur le mécanisme de sélection naturelle du vivant et notamment sur le concept de mutation. 12

remédiation La remédiation est le fait de supprimer une vulnérabilité d'un programme par la modification de son code. 1, 7, 14, 17, 19, 26, 27, 29, 30, 32, 36, 37, 39

renforcement Le renforcement (*Hardening* en anglais) est un ensemble de remédiation préventive afin de protéger un programme d'attaques et de failles de sécurité potentielles. 14–16

root Le *root* est la pratique d'élévation de privilèges sur un système *Android* permettant d'obtenir les droits d'administrateur sur l'appareil mobile. 7, 15, 17, 22, 37

réparation automatisée de programme La réparation automatisée de programme (*Automated Program Repair* abrégé **APR**) est un domaine de l'informatique qui a pour but de corriger de façon automatique les bugs sans modifier le fonctionnement d'un programme. 8

réseau de neurones artificiels Un réseau de neurones artificiels est un système basé sur le fonctionnement des neurones biologiques ayant dérivés vers une base statistique. 11, 13

réseau neuronal convolutif Un réseau neuronal convolutif (*Convolutional Neural Network* abrégé **CNN**) est une catégorie de réseau de neurones artificiels appliqué à l'analyse visuelle d'image basé sur le produit de convolution. 13

rétro-ingénierie La rétro-ingénierie est la pratique ayant pour objectif de déduire le fonctionnement interne d'un programme à partir de son code, que ce soit un code source, compilé ou obfusqué. 7, 14, 15

rétropropagation du gradient La rétropropagation du gradient (*Back Propagation* abragé **BP**) est une méthode pour entraîner un réseau de neurones artificiels, consistant à mettre à jour les poids de chaque neurone de la dernière couche vers la première. 11

réusinage Le réusinage (*Refactoring* en anglais) est une pratique visant à utiliser des patrons afin de modifier un code source sans altérer son fonctionnement dans le but de le rendre plus lisible et/ou plus simple. 13

traduction automatique neuronale La traduction automatique neuronale (*Neural Machine Translation* abrégé **NMT**) est une approche de la traduction automatique utilisant les réseau de neurones artificielss pour prédire des suites de mots. 13

traitement automatique des langues Le traitement automatique des langues (*Natural Language Processing* abrégé **NLP**) est un domaine de l'intelligence artificielle ayant pour objectif de comprendre le langage naturel de la même façon qu'un humain. 13

usurpation L'attaque par usurpation (*Spoofing* en anglais) est une attaque où un attaquant envoie des données incorrectes à une autre en se faisant pas pour la source souhaité. 20

Acronymes

API interface de programmation. 16, 17, 20, 23, 25–27, 29, *Glossaire :* interface de programmation

APR réparation automatisée de programme. 8–10, 12–14, 36, 39, *Glossaire :* réparation automatisée de programme

AST arbre de la syntaxe abstraite. 12, 13, 30, 34, *Glossaire :* arbre de la syntaxe abstraite

BP rétropropagation du gradient. 11, *Glossaire :* rétropropagation du gradient

CNN réseau neuronal convolutif. 13, *Glossaire :* réseau neuronal convolutif

FCA analyse formelle de concepts. 12, *Glossaire :* analyse formelle de concepts

FL Localisation des défauts (*Fault Localization* en anglais). 9, 10, 13

GP programmation génétique. 12, *Glossaire :* programmation génétique

IoT L'internet des objets (*Internet of Things* en anglais) est l'interconnexion entre internet et les différents objets connectés. 7

LLM grand modèle de langage. 13, *Glossaire :* grand modèle de langage

LTL logique temporelle linéaire. 12, *Glossaire :* logique temporelle linéaire

NLP traitement automatique des langues. 13, *Glossaire :* traitement automatique des langues

NMT traduction automatique neuronale. 13, *Glossaire :* traduction automatique neuronale

OS Système d'exploitation (*Operating System* en anglais). 4, 5, 15, 24

PG Génération de correctifs (*Patch Generation* en anglais). 9, 12, 13

PV Validation de correctifs (*Patch Validation* en anglais). 9, 13

RASP auto-protection des applications en exécution. 8, *Glossaire :* auto-protection des applications en exécution

RBF fonction de base radiale. 11, *Glossaire :* fonction de base radiale

RNN réseau de neurones artificiels récurrents. 13, *Glossaire :* réseau de neurones artificiels récurrents

SDK kit de développement. 28, *Glossaire :* kit de développement

Table des matières

Table des matières

1 Introduction **1**

2 État de l'art **3**
2.1 Introduction 3
2.2 Concepts et Taxonomie du Domaine de Recherche 3
2.3 Travaux Connexes 10
2.4 Problèmes et Question de Recherche 18

3 Contribution **19**
3.1 Introduction 19
3.2 Sélection et Détection des Vulnérabilités 19
3.3 Remédiation des Vulnérabilités 26
3.4 SHIELD : Un outil pour une réalisation automatique du blindage d'applications mobiles Android 30
3.5 Réponse à la Question de Recherche 36

4 Perspectives **37**

5 Conclusion **39**

Bibliographie **41**

Annexes **51**

1

Introduction

La donnée est une ressource cruciale et son stockage numérique s'est largement développé ces dernières années. Qu'elle appartienne à des états, des entreprises ou des particuliers, elle est la cible de nombreux attaquants, que ce soit à des fins malveillantes, d'espionnage ou même monétaire[1].

La sécurité en informatique a eu un essor important au cours des dernières années, corrélé à celui de la démocratisation de l'informatique. Les supports informatiques sont partout de nos jours et cette grande diversité d'appareils entraine une d'autant plus grande diversité des vulnérabilités. La sécurité informatique s'est alors diversifiée en plusieurs branches, chacune spécialisée dans un domaine, afin de répondre à ce besoin en versatilité.

La sécurité logicielle se concentre sur la sécurisation des divers logiciels et de leurs codes. Nous allons nous pencher sur son application au support mobile qui est au cœur de nos vies. Que ce soit pour une utilisation professionnelle ou personnelle, les appareils mobiles contiennent des informations confidentielles qui peuvent attirer des attaquants.

De nombreuses méthodes permettent d'améliorer la sécurité des applications, mais si l'attaquant a connaissance de ces méthodes, il peut plus facilement les contourner[2]. C'est pour répondre à cette problématique que la méthode du blindage a été développée[3]. Elle permet de complexifier la tâche de l'attaquant en rendant l'application la plus hermétique possible. Cette approche permet donc de réduire les attaques en augmentant les ressources nécessaires afin de la mener à bien[4] et de se protéger du vol de données en complexifiant leur lecture[5].

Nous allons donc dans un premier temps nous pencher sur la compréhension de ce domaine, ainsi que sur une analyse des techniques en vigueur. Dans un second temps, nous allons présenter notre contribution au domaine du blindage d'applications mobiles *Android* à travers une approche du blindage basée sur des patrons associant vulnérabilité et remédiation et de sa validation à travers l'outil automatisé **SHIELD**. Nous terminerons ce rapport en présentant les perspectives en lien avec notre contribution et le domaine de recherche.

2

État de l'art

2.1 Introduction

Afin de présenter le contexte de ce travail de recherche et son cadre d'application, nous allons d'abord présenter une taxonomie du domaine de recherche qui présente les différents concepts clés. Ensuite, nous allons présenter les différents travaux connexes afin d'évaluer l'avancée des solutions proposées dans le domaine.

Dans notre contexte, les termes : application, logiciel et programme, sont interchangeables afin de définir un composant applicatif informatique indépendant. Il en va de même pour les termes : appareil, machine et ordinateur qui définissent le support physique d'exécution. Enfin, les termes vulnérabilité, menace et faille définissent tous les trois un problème de sécurité.

2.2 Concepts et Taxonomie du Domaine de Recherche

2.2.1 Sécurité Logicielle

Le concept de la sécurité peut être appliqué à de nombreux domaines. Pour cette taxonomie, nous allons nous concentrer sur son application à l'informatique puis, plus particulièrement, à son utilisation au niveau logiciel.

2.2.1.1 Type de Sécurité Informatique

La sécurité informatique[6] est l'application du concept de la sécurité au domaine de l'informatique. Ce domaine, synthétisé à la figure 2.1, peut se décomposer en plusieurs sous-domaines selon l'aspect à sécuriser.

On retrouve tout d'abord la sécurité physique telle qu'elle est décrite par Weis [7]. Chaque appareil informatique présente une interface physique à travers des ports en façade, mais aussi à travers les interfaces des différentes cartes électroniques. Chacune de ces connexions peut représenter une menace si elle est utilisée à mauvais escient. Le cas le plus connu est le détournement du port *USB*[8] ou encore des enregistreurs de frappes[9].

Un autre aspect de la sécurité informatique est la sécurité des infrastructures, comme indiqué par Abouzakhar [10]. Cette branche se concentre sur la configuration et le matériel employés afin de mettre en place une infrastructure. On retrouve des aspects liés à la gestion des utilisateurs et des droits ainsi que des postes de travail. Mais une grande partie de cet aspect se concentre sur la partie réseau de l'infrastructure afin de limiter chaque poste aux accès qui lui sont nécessaires. Tout en empêchant les intrusions externes et les élévations de privilèges en interne.

La sécurité informatique se compose aussi d'une branche système qui se concentre sur les systèmes d'exploitation(*Operating System* abrégé **OS**OS tel que présenté par Hamdani et al. [11]. Étant le composant logiciel principal d'un ordinateur, une branche de la sécurité informatique lui est totalement dédiée. Il se distingue des autres logiciels par l'importance qu'il a au sein d'une machine. En effet, il gère à la fois l'aspect matériel de la machine et les utilisateurs qui travaillent dessus. L'exploitation de bugs au sein d'un OS peuvent avoir des conséquences critiques en donnant un accès complet à un attaquant.

Une autre branche de la sécurité informatique est la sécurité réseau comme avancé par Kumar [12]. Au contraire de la sécurité de l'infrastructure qui s'occupe de la mise en place matérielle du réseau, la sécurité réseau se concentre sur la transmission des données et leur contenu. L'exemple du cas du *TCP/IP* est présenté par Harris and Hunt [13]. La sécurité réseau peut intervenir sur la grande majorité des couches du modèle OSI[14]. Un exemple de sécurité réseau visible par le grand public est l'utilisation du protocole *HTTPS* en remplacement au protocole *HTTP*.

La sécurité informatique possède une branche plus récente, définie par Wang et al. [15], à mi-chemin avec les sciences sociales. La branche sociale englobe toutes les vulnérabilités utilisant l'humain pour pénétrer un logiciel, un réseau ou une infrastructure[16, 17]. Le cas le plus courant d'attaque dans cette catégorie est l'hameçonnage qui se base sur l'inattention ou la crédulité de l'utilisateur afin de le faire saisir des informations confidentielles dans un contexte corrompu. Ce type d'attaque a été défini et analysé par Aleroud and Zhou [18].

On retrouve enfin la sécurité logicielle qui est une branche dédiée à la sécurisation des applications dont Amoroso [19] en montrent l'évolution. C'est la branche associée au génie logiciel. Cet aspect englobe un grand nombre de pratiques qui touche une diversité de métiers, du développeur au redistributeur de logiciel. Son niveau de criticité a fortement augmenté ces dernières années avec la démocratisation de l'informatique dans la population et son utilisation intensive dans tous les domaines. La protection des données personnelles face au vol et à la revente est devenue une préoccupation majeure. Dans ce cadre, les pratiques sont diverses et peuvent être catégorisées selon plusieurs critères, comme c'est montré dans les sections ci-dessous.

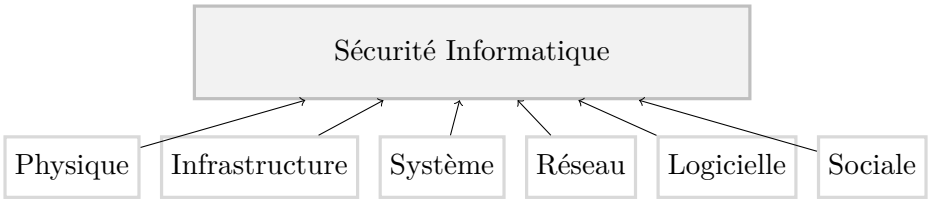


FIGURE 2.1 : Types de Sécurité Informatique

2.2.1.2 Cible de la Sécurité Logicielle

Un critère de catégorisation des pratiques de sécurité logicielle est la cible. En effet, les logiciels possèdent des supports très variés. Chaque support possède ses propres vulnérabilités et la sécurité doit donc s'en adapter. Hamdani et al. [11], dans leur comparaison, présentent des différences significatives selon les OS. Tout comme Babate et al. [20] présentent des différences entre support de bureau et mobile. Une agrégation de ces cibles est présentée à la figure 2.2.

On retrouve tout d'abord les applications de bureaux qui sont exécutées par des ordinateurs personnels. Elles se distinguent des applications dorsales qui sont généralement exécutées sur des serveurs. On retrouve ensuite les applications mobiles qui sont exécutées par un OS mobile. Enfin, on distingue les applications web qui sont exécutées par un navigateur en s'abstrayant du support physique.

Cette distinction se base sur le fait que les appareils ont une interface physique qui peut grandement varier et donc présenter des risques différents pour l'application. De plus, pour chacun de ces supports physiques, l'OS utilisé peut aussi varier et donc présenter à nouveaux d'autres risques.

Pour les applications bureaux, on retrouve alors trois sous-catégories correspondant aux OS présents sur le marché qui sont *Windows*, *MacOS* et les OS respectant le standard *Unix*. Les applications dorsales, en comparaison, ne possèdent que deux sous-catégories : *Windows* et les OS du standard *Unix*. Pour les applications mobiles, on retrouve trois sous-catégories qui sont *Android*, *iOS* et le multiplateforme. Ce dernier, présenté par Delia et al. [21], regroupe les différents cadriciels (*Flutter* par Google [22], *Xamarin/.NET* par Microsoft [23], ...) qui permettent d'utiliser un même code pour exporter vers différents OS du marché ou le web. Ces cadriciels rajoutent une surcouche afin de s'abstraire des OS ciblés. Ils possèdent donc leurs vulnérabilités propres à cette surcouche. Pour les applications web, la standardisation des technologies telles que *JavaScript* avec *ECMAScript*[24] réduit considérablement les différences d'implémentation entre les différents navigateurs. Il est alors moins pertinent de différencier la sécurisation de telles applications selon la plateforme ciblée.

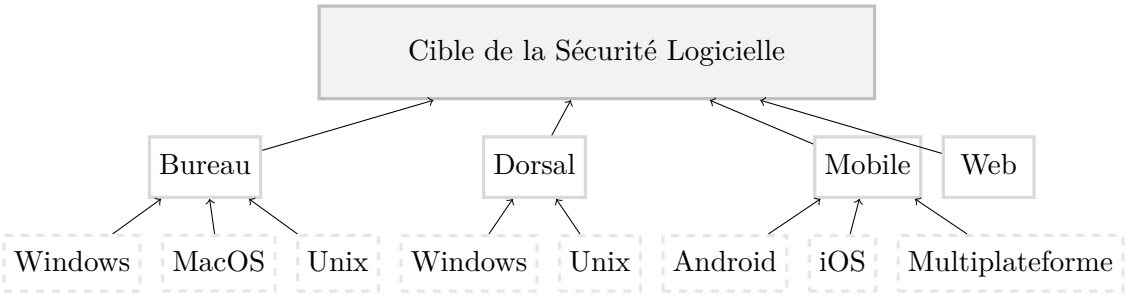


FIGURE 2.2 : Cibles de la Sécurité Logicielle

2.2.1.3 Temporalité de la Mise en Place de la Sécurité Logicielle

Un autre critère de catégorisation du domaine de la sécurité logicielle est la temporalité de sa mise en œuvre. Le cycle de développement et de déploiement d'une application est complexe[25], et le logiciel passe par plusieurs états dans lesquels il est possible d'intervenir afin de le sécuriser. Ces différentes temporalités sont synthétisées à la figure 2.3. Mantovani et al. [26] mettent en avant les différences entre l'état "source" et l'état "empaqueté" dans la détection de bugs. Bien qu'il ne s'agisse pas de problèmes de sécurité dans l'étude qu'ils ont mené, la démarche de détection de bugs et de faille est similaire par sa démarche d'analyse et de compréhension du code afin d'y détecter un défaut. Les difficultés rencontrées sont donc aussi similaires.

L'état "source" est celui où se trouve l'application durant les phases de développement. Dans cet état, le code est facilement compréhensible par un être humain. Durant ces phases, la sécurité logicielle prend la forme de bonnes pratiques à employer par les développeurs[27], de recommandations faites sur les vulnérabilités dans le code[28], ou sur l'utilisation d'outils externes de modification de code[29].

Le second état est "empaqueté". On qualifie d'empaqueté une application prête à être déployée. Elle n'est pas simplement sous la forme d'un paquet. Elle peut être est le résultat d'un ou plusieurs optimiseurs, compilateurs, transpileurs et empaqueteurs afin de la rendre utilisable en production. La forme réelle de l'application est fortement dépendante des technologies utilisées et de la plateforme cible. Dans cet état, le code est généralement difficilement lisible par un être humain et plutôt dédié à une machine physique ou virtuelle. La sécurité logicielle est souvent liée à des approches implémentées par des outils externes de l'application afin de permettre la modification de code.

Le troisième et dernier état possible est "en exécution". C'est dans cet état qu'est le logiciel lorsqu'il est en cours d'utilisation sur une machine. La sécurité logicielle prend alors la forme de sécurité interne à l'application, ou alors d'une application externe d'observation et de contrôle de l'exécution.

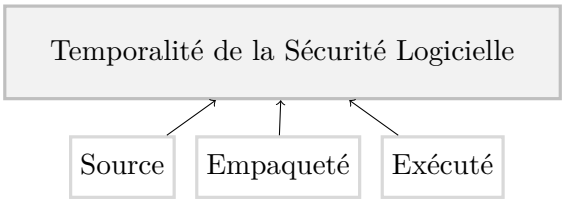


FIGURE 2.3 : Temporalité de la Sécurité Logicielle

2.2.1.4 Action de la Sécurité Logicielle

Un dernier critère de catégorisation de la sécurité logicielle est le type d'action menée pour se défendre. Ces actions peuvent être réalisées de façon automatique ou manuelle. Une synthèse est montrée sur la figure 2.4.

On distingue tout d'abord la détection de maliciels, *Malware* en anglais, qui sont des logiciels volontairement malveillants. C'est un domaine à part entière comme le montre Naseer et al. [30]. Elle vise à protéger les applications en détectant et traitant les menaces externes comme le présente Pandey et al. [31].

On peut ensuite citer la détection de vulnérabilités présentée par Qasem et al. [32]. Cette détection peut se faire sous deux angles. Le premier est l'angle interne qui consiste à détecter des vulnérabilités laissées dans le code de l'application. Le second est l'angle externe qui vise à détecter des vulnérabilités dans l'environnement d'exécution. C'est-à-dire des faiblesses pouvant présenter un risque pour l'intégrité du système face à une attaque. Cela peut être au niveau de sa configuration avec l'utilisation de débugueur, ou de son état, comme les appareils root par exemple[5]. Ce second angle est bien évidemment limité aux sécurités appliquées lors de l'exécution (cf. section 2.2.1.3) puisqu'il est nécessaire de posséder un environnement afin de pouvoir y détecter des vulnérabilités. De plus, il s'applique particulièrement aux applications clients pour lesquels l'environnement est inconnu et n'est pas maitrisé. En effet, il est alors nécessaire d'analyser cet environnement pour y déceler des failles et agir en conséquence lors de l'exécution étant donné que celles-ci ne peuvent pas être connues en avance. Au contraire des applications dorsales par exemple qui s'exécutent dans un environnement connu et mis en place par le développeur. On retrouve ce second point dans la présentation de Haupert et al. [3].

Une autre action de la sécurité logicielle est la remédiation. Cette approche a pour objectif de modifier le code d'une application afin d'en retirer les vulnérabilités. Ashraf and Habaebi [33] appliquent cette approche au domaine de l'internet des objets (*Internet of Things* abrégé **IoT**). Elle s'appuie sur la détection de vulnérabilités citées précédemment. Deux angles peuvent être choisis pour réaliser cette approche. Le premier est la correction de vulnérabilités existantes où une vulnérabilité est présente dans le code et doit en être retirée. Des exemples de ce type de remédiation sont proposés par Graff and Van Wyk [27]. La seconde est la correction préventive qui consiste à empêcher de possibles attaques en renforçant l'application et en empêchant la rétro-ingénierie. Ce second angle est appelé blindage dans le contexte des applications mobiles et est étudié par Haupert et al. [3]. Le blindage, *Shielding* en anglais, est donc un ensemble de méthodes ayant pour objectif de permettre l'exécution d'une application dans un milieu hostile en l'ayant protégé à l'aide de méthodes préventives.

Une autre action, qui est spécifique à l'état d'exécution d'une application, est nommée auto-protection des applications en exécution (RASP) et a été défini par Čisar and Čisar [34]. Elle a pour objectif de protéger une application durant son exécution. Elle peut soit intégrer des fonctions de sécurité supplémentaire qui vont être exécutés en même temps que l'application. Soit utiliser une application externe afin d'observer l'application à sécuriser durant son exécution.

Enfin, un dernier type d'action, dans le cadre de la sécurité logicielle, est l'utilisation des tests, comme présenté par Potter and McGraw [35]. Cette approche utilise des suites de test effectuant des vérifications de sécurité. Elles prennent la même forme que celles des tests unitaires ou d'intégration. Pour cette même raison, ils en possèdent les avantages et les inconvénients. Ces tests peuvent être réalisés en boite blanche, grise ou noire selon le niveau de connaissance, possédé ou souhaité, du fonctionnement interne de l'application[36]. Les tests en boite blanche se font avec une connaissance complète du programme et permettent d'en tester le fonctionnement interne. Au contraire des tests en boite noire qui se font sans connaissance du fonctionnement interne, et ne vérifie que l'aspect externe du programme et notamment les sorties de données produites. Enfin, les tests en boite grise allient les deux techniques précédentes.

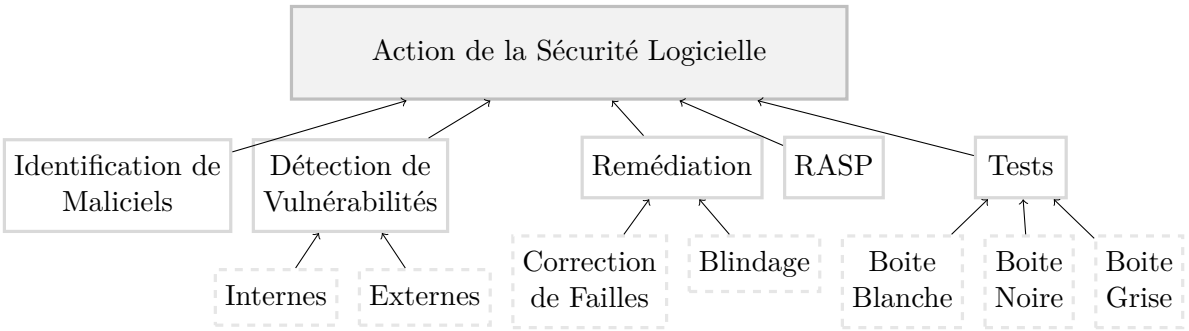


FIGURE 2.4 : Actions de la Sécurité Logicielle

Une agrégation de différentes classifications précédentes est visible en annexe 1 afin de présenter une vue d'ensemble du domaine.

2.2.2 Réparation Automatisée de Programme

La réparation automatisée de programme (APR), de l'anglais *Automated Program Repair*, est un domaine visant à corriger les défauts d'un programme automatiquement sans en modifier les fonctionnalités.

Dans le cadre du domaine de l'APR, les défauts sont assimilés aux bugs. Mais la notion de défaut peut être généralisée afin d'englober un plus grand nombre de concepts sous ce nom. On peut par exemple définir un défaut comme étant un problème de qualité du code ou, comme dans notre cas, une vulnérabilité logicielle.

C'est dans l'optique de cette généralisation que nous souhaitons étudier les approches de l'APR afin de s'en inspirer et de les réutiliser dans le contexte de l'automatisation de l'application du blindage.

Huang et al. [37] en définissent les concepts. Les méthodes utilisées peuvent se généraliser en un processus en trois étapes. Dans un premier temps, il est nécessaire de localiser les défauts à corriger, puis de générer le correctif, pour enfin valider ce correctif sur son efficacité ainsi que la non-régression du programme.

2.2.2.1 Localisation des Défauts

La localisation des défauts (*Fault Localization* abrégé **FL**) est la première étape de l'APR comme décrit par Wong et al. [38]. Elle consiste à localiser, avec plus ou moins de granularité, un défaut. La granularité peut aller d'une instruction à un fichier de code. Certaines méthodes de localisation sont fortement couplées avec la méthode de génération de correctifs (*Patch Generation* abrégé **PG**), c'est généralement le cas des méthodes utilisant l'apprentissage automatique. Tout comme les deux autres étapes, elle joue un rôle critique. Un faux positif lors de la localisation peut entrainer une modification du code inutile, voire l'insertion de nouveaux défauts. Un faux négatif est d'autant plus problématique parce qu'il laisse un défaut réel dans le programme.

2.2.2.2 Génération de Correctifs

La PG est au centre de la démarche de l'APR. Elle est particulièrement détaillée par Huang et al. [37]. Elle prend en entrée les défauts localisés par l'étape précédente et génère un ou des correctifs candidats. L'ensemble des correctifs candidats est large et en générer un certain nombre peut être chronophage. Toute la criticité de cette tâche, en plus de produire un candidat valide, est de le faire en un temps raisonnable. Une attention toute particulière à la précision et à l'optimisation est à porter durant la réalisation de cette tâche.

2.2.2.3 Validation de Correctifs

La validation de correctifs (*Patch Validation* abrégé **PV**) est la dernière étape. Elle prend en entrée les différents correctifs générés et vérifie s'ils sont corrects et s'ils corrigent bien le défaut souhaité. Une analyse de cette étape est réalisée par Liu et al. [39]. Tout comme la vérification fonctionnelle des programmes, c'est une tâche ardue à réaliser et c'est la méthode des tests qui est la plus largement répandue. Dans les deux cas, ils présentent les mêmes inconvénients, leur manque d'exhaustivité ainsi que le besoin de les rédiger manuellement. Les méthodes de preuves formelles permettent de se libérer du manque d'exhaustivité, mais demandent bien plus de compétences et de temps.

2.2.3 Positionnement par Rapport au Domaine de Recherche

Après avoir présenté les éléments du contexte de notre travail de recherche, il est maintenant possible de positionner le cadre de ce travail.

Celui-ci s'inscrit dans le cadre de la sécurité logicielle ciblant des tierces-applications mobiles. La notion de tierce-application induit un état empaqueté de celle-ci. Cet état est, comme indiqué précédemment, fortement dépendant de la plateforme. Notre choix s'étant porté vers *Android* pour son parc d'appareils en service et son noyau ouvert, les

applications seront sous forme d'APK. Enfin, les besoins de sécurité et confidentialité trouvent une réponse adaptée à travers les méthodes de blindage

L'objectif étant de remédier de façon automatisée à ces vulnérabilités, l'APR devient un domaine connexe proche concernant les techniques de modification automatique du code.

2.3 Travaux Connexes

Nous présentons dans cette section des travaux que nous considérons connexes par rapport à l'objectif général de notre travail. Il s'agit donc de travaux liés à l'APR et le blindage des applications logicielles.

2.3.1 Réparation automatisée de programme

Le domaine de l'APR est découpé en trois parties par Huang et al. [37] comme décrit précédemment. Les techniques proposées vont être présentées selon l'ordre du processus d'APR

2.3.1.1 Localisation des Défauts

Wong et al. [38] distinguent deux types de FL. La première est nommée méthodes traditionnelles et regroupe les méthodes manuelles qu'un développeur peut réaliser, comme l'ajout d'affichage pour des variables du programme ou l'utilisation de débugueur. La seconde sorte de méthodes est nommée méthodes avancées. C'est celle-ci qui est utilisée dans le contexte de l'APR. Cette seconde classe de méthodes est divisée en plusieurs familles par Wong et al. [38], regroupées selon l'approche utilisée.

Approches basées sur les tranches. Les approches basées sur les tranches, *Slice* en anglais, visent à réduire la recherche de défauts en découpant un programme en tranches. Chaque tranche est un ensemble d'instructions qui, pour une instruction donnée, contient toutes les instructions pouvant modifier une variable de l'instruction d'origine. Dans le cas des tests unitaires, un test échoue généralement à cause d'une variable ayant une valeur incorrecte, il suffit donc de chercher l'erreur dans la tranche associée à cette variable.

Une construction de tranches par analyse statique a été proposée par Weiser [40]. Lyle [41] a étendu cette technique à la création de découpe, *dice* en anglais, qui correspond à l'ensemble des instructions de la différence entre deux tranches afin de réduire le domaine de recherche. Des constructions de tranche par analyse dynamique ont été proposées par Agrawal et al. [42] ou encore Zhang et al. [43]. Une dernière alternative a été proposée par Agrawal et al. [44] afin de construire des tranches à partir de l'exécution de tests en incluant dans la tranche l'ensemble des instructions exécutées par un test.

Approches basées sur le spectre du programme. Les approches basées sur le spectre du programme, *Program Spectrum* en anglais, utilisent le spectre comme étant une structure contenant des informations sur l'exécution du programme tel que les informations sur les branches conditionnelles ou les fils d'exécution internes aux procédures.

Dans un premier temps, des études comme celle de Agrawal et al. [45] utilisent uniquement les spectres de tests échoués. Des cas plus avancés font utilisation du *Executable Statement Hit Spectrum (ESHS)* qui enregistre l'ensemble des instructions exécutables exécutées. Renieres and Reiss [46] l'utilisent afin de réaliser des opérations sur des ensembles d'instructions appartenant à des tests réussis ou échoués, ou encore afin de parcourir un graphe de dépendances par ordre de plus proche voisin. Jones and Harrold [47] utilisent l'*ESHS* afin de calculer des métriques telles que la couverture des tests et les résultats de leur exécution afin d'en obtenir un indice de *suspicion* pour les instructions.

Approches basées sur les statistiques. Les approches basées sur les statistiques se basent sur des prédicats associés à différents points du code. Les statistiques sur la valeur de ces prédicats durant les tests réussis et échoués sont ensuite utilisées pour la détection.

Liblit et al. [48] utilisent cette approche en calculant les probabilités qu'un prédicat vérifié indique un défaut et les probabilités qu'un même prédicat soit exécuté dans le cas d'un défaut afin de calculer un score d'importance par paire prédicat-défaut. Liu et al. [49] calculent la probabilité qu'un prédicat soit vrai à chacune de ses évaluations et comparent sa distribution entre les exécutions de tests réussis et échoués afin d'en déduire un score de suspicion. Wong et al. [50] proposent une approche basée sur la CROSS-CLASSIFICATION TABLE afin de comparer la couverture et le succès des tests afin d'obtenir un score de suspicion.

Approches basées sur l'état du programme. Les approches basées sur l'état du programme utilisent une image des variables en mémoire et de leurs valeurs à un point de l'exécution du programme, *Program State* en anglais.

Zimmermann and Zeller [51] utilisent cette image afin de comparer la mémoire dans le cas de test échoué et réussi et en remplaçant des valeurs de tests réussis par des tests échoués afin de vérifier leur impact sur le succès du test. Zhang et al. [52] se basent sur l'ajout de prédicat dont leur évaluation positive indique une exécution réussie. Ils modifient ensuite l'état du programme en modifiant la valeur des variables jusqu'à faire échouer les prédicats. Wang and Roychoudhury [53] modifie l'état de différentes branches du programme lors de l'exécution de tests échoués, si le test réussi, la branche est marquée comme défaillante.

Approches basées sur l'apprentissage. Les approches basées sur l'apprentissage appliquent les techniques d'apprentissage automatique à la détection de défauts.

Parmi les utilisateurs des réseaux de neuronnes artificiels, Wong and Qi [54] basent leur approche sur les réseaux à rétropropagation du gradient (BP) tandis que Wong et al. [55] orientent leur approche sur les réseaux à fonction de base radiale (RBF). On retrouve aussi Briand et al. [56] qui utilisent l'algorithme C4.5 DECISION TREE ALGORITHM qui est un algorithme de génération d'arbre de décision.

Approches basées sur les modèles. Cette approche se base sur des modèles de différentes natures selon la méthodologie choisie. Ils sont un sujet de recherche actif afin de produire des modèles efficaces.

DeMillo et al. [57] utilisent des relations préétablies manuellement entre la source d'un échec et les types d'échec existant afin d'en trouver l'origine. Wotawa et al. [58] proposent un modèle construit à partir d'une analyse statique afin de représenter un programme sous la forme de la logique du premier ordre et compare l'état du programme lors de l'échec d'un test avec celui du modèle pour y trouver des différences.

Approches basées sur l'exploration de données. De façon similaire aux approches basées sur l'apprentissage, l'exploration de données, *Data Mining* en anglais, vise à extraire un modèle et un ensemble de règles à partir d'un jeu de données. Les techniques d'explorations de données permettent une détection d'information qu'un humain n'aurait pas pu réaliser, mais aussi de traiter un plus grand nombre de données.

Nessa et al. [59] extraient des N-grammes à partir du code et utilisent la probabilité qu'un certain N-gramme apparaisse dans la trace d'un test échoué pour déterminer les instructions pouvant contenir un défaut. Cellier et al. [60] déduisent des règles d'association entre une instruction et un test échoué, et utilise des techniques d'analyse formelle de concepts (FCA) pour les explorer.

2.3.1.2 Génération de correctifs

Huang et al. [37] présentent quatre grandes familles de PG : les approches basées sur la recherche, les contraintes, l'apprentissage ou les modèles. Certaines méthodologies peuvent allier plusieurs approches comme celle proposée par Long and Rinard [61] qui allie les approches basées sur les patrons, la recherche et les contraintes.

Approches basées sur la recherche. Les approches basées sur la recherche utilisent des heuristiques afin d'explorer le plus efficacement l'espace de recherche contenant les correctifs possibles.

Jobstmann et al. [62] proposent une méthode souhaitant se rapprocher du jeu de stratégie. Elle se compose d'une version modifiable du programme et d'un automate représentant sa spécification en logique temporelle linéaire (LTL). Chaque état gagnant du jeu correspond à une réparation réussie. Le Goues et al. [63] utilisent la programmation génétique (GP) sur un arbre de la syntaxe abstraite (AST) afin de produire des mutations au niveau des instructions et de trouver un AST correct. L'application de la GP à l'APR est encore un sujet actif de recherche. Ji et al. [64] se basent sur un indice de similarité entre différent codes afin de modifier l'AST d'un programme défaillant à l'aide de celui d'un programme fonctionnel.

Approches basées sur les contraintes. Les approches basées sur les contraintes ont pour objectif de transformer la recherche d'un correctif en la résolution d'un réseau de contraintes.

Wei et al. [65] utilisent les contrats afin de diriger leur processus d'APR. Nguyen et al. [66] basent leur approche sur l'utilisation de l'exécution symbolique et d'une suite de tests afin de générer le réseau de contraintes. Le et al. [67] ont pris le parti d'ajouter à la GP des méthodes de VÉRIFICATION DÉDUCTIVE. Van Tonder and Goues [68] proposent une génération de spécification formelle à l'aide de la LOGIQUE DE SÉPARATION qui est une extension de la LOGIQUE DE HOARE.

Approches basées sur l'apprentissage. Les approches basées sur l'apprentissage utilisent des techniques d'apprentissage automatique afin de générer un correctif. Ces approches ont eu une forte croissance durant ces dernières années et sont un domaine particulièrement actif, notamment avec l'émergence des techniques d'apprentissage profond. Les approches basées sur l'apprentissage allient généralement à la fois les étapes de FL et PG. La nature de leur fonctionnement permet à la fois de construire des modèles de détection et de correction.

Tufano et al. [69] utilisent une traduction automatique neuronale (NMT), et donc des techniques de traitement automatique des langues (NLP), en considérant le code sous sa forme textuelle. La tâche de correction de bug est alors perçu comme la traduction d'une instruction défaillante vers une instruction correcte. White et al. [70] se basent eux sur l'utilisation de l'apprentissage profond, qui utilise du code correct afin d'en déduire des correctifs pour l'étape de correction. Li et al. [71] proposent une utilisation de l'apprentissage profond différente en travaillant non pas sur le code sous sa forme textuelle, mais sur l'AST. Ye et al. [72] ont développé un nouveau modèle qui, au contraire des modèles de NLP, est dédié à l'APR et non au langage de façon globale. Xia and Zhang [73] basent leur approche sur des modèles plus récents nommés grand modèle de langage (LLM)s. Gupta et al. [74] utilisent une approche mixte en utilisant un réseau de neurones artificiels récurrents (RNN) pour la FL et l'apprentissage profond pour la PG.

Approches basées sur les patrons. Les approches basées sur les patrons se servent de patrons préétablis pour créer de façon systématique un correctif. C'est une approche qui peut se rapprocher des méthodes de réusinage de code tout en ayant des objectifs bien distincts. Cette approche est elle-même divisée en deux sortes. D'un côté, on distingue l'extraction manuelle de patron, c'est-à-dire que les patrons de correctifs sont définis par un humain. D'un autre côté, on retrouve l'extraction automatique qui consiste à laisser l'outil déduire lui-même les caractéristiques du patron de correctif. Tout comme pour les approches basées sur l'apprentissage, les approches basées sur les patrons s'occupent à la fois de l'étape de FL et de PG.

Kim et al. [75] proposent une solution ou les patrons, au nombre de dix, sont extraits manuellement à partir de projets open-source. Tan and Roychoudhury [76] ont développé une méthode de non-régression en utilisant l'historique de changement de soixante-treize logiciels. Le et al. [77] utilisent, eux aussi, l'historique de changement, mais allié à une extraction automatique des patrons. Liu et al. [78] se basent sur un réseau neuronal convolutif (CNN) pour l'extraction automatisée. Cette approche sera ensuite améliorée par Liu et al. [79] qui utilise des outils externes pour la FL. Koyuncu et al. [80] extraient leurs patrons à partir des rapports de bugs.

2.3.1.3 Validation de correctif

Cette dernière étape nommée PV est encore à l'étude. Aucune technique totalement fiable ne fait consensus. La méthode de vérification phare réside dans l'utilisation des suites de tests réalisées pour la vérification fonctionnelle du programme. Certains projets se basent sur les contrats pour cette étape. Dans le cadre actuel, la métrique la plus

utilisée pour évaluer les APRs est le nombre de défauts corrigés, bien que Liu et al. [39] remettent en cause sa pertinence seule.

2.3.2 Blindage

Le blindage est un terme marketing à l'origine pour décrire un service de protection ajouté à une application existante, notamment dans un contexte de propriété intellectuelle, comme l'explique par Newman [81]. Cette démarche s'est développée pour englober des sécurités sur l'intégrité applicative et la lutte face à la rétro-ingénierie afin d'augmenter l'effort nécessaire lors d'une attaque tel que dit dans la citation de Zumerle [4]. Des études comme celle de Jung et al. [2] montrent qu'il est possible d'empaqueter à nouveau des applications afin de détourner leur fonctionnement d'origine, ce qui est un exemple de l'utilité de la remédiation préventive.

Les sécurités mises en place dans le cadre du blindage sont suffisamment diversifiées pour être classées en deux catégories. On retrouve d'un côté l'obfuscation, incluant les chiffrements, et d'un autre le renforcement.

2.3.2.1 Obfuscation

L'obfuscation est un ensemble de techniques, plus générales que le blindage, visant à complexifier la lecture et la compréhension du code. Elles sont ici employées afin de rendre plus difficile à un attaquant la rétro-ingénierie. Faruki et al. [82] présentent l'ensemble de l'historique des méthodes d'obfuscation qui ont été et qui sont encore actuellement utilisées dans le cas d'*Android*.

Dans cet article, on retrouve tout d'abord la modification du flot d'exécution :

- En insérant du code mort
- En complexifiant les boucles
- En ajoutant de la redondance
- En insérant du parallélisme
- En remplaçant l'appel de méthodes par leur corps
- En utilisation la réflexivité

On retrouve aussi la complexification des valeurs :

- En calculant les valeurs littérales
- En modifiant les classes
- En divisant les variables puis en les reconstruisant avec des opérateurs binaires
- En réordonnant l'initialisation des variables

Enfin, il est possible de modifier la structure même du code en modifiant totalement les noms de variables, méthodes ou classes.

En plus de l'obfuscation traditionnelle, le *shielding* ajoute le chiffrement de certaines informations. Il est par exemple pertinent de chiffrer les chaines de caractères écrites en dur dans le code, les ressources externes empaquetés dans l'application ou encore directement les binaires exécutables.

2.3.2.2 Renforcement

Le renforcement est un ensemble de pratiques visant à protéger l'intégrité du fonctionnement d'un logiciel.

C'est un concept qui a déjà été utilisé pour des OS afin de s'autoprotéger de bugs comme l'indique Mourad et al. [83]. Son adaptation au support mobile et à la sécurité logicielle est plus récente et s'est d'abord développés dans le secteur des sociétés de sécurité privée comme Promon [84]. Mais c'est Haupert et al. [3] qui a, à notre connaissance, pour la première fois en 2018 défini la pratique du renforcement.

Cette approche est fortement dépendante de la plateforme sur laquelle elle est réalisée, car chaque plateforme à ses propres vulnérabilités, ses propres risques et sa propre architecture. Le support mobile est d'autant plus vulnérable qu'il contient des informations critiques et personnelles. Seul Sihag et al. [5] et Haupert et al. [3] abordent le sujet dans le contexte d'une application *Android*.

Haupert et al. [3] définissent le renforcement comme étant un ensemble de méthodes permettant d'assurer la sécurité d'une application même sur des OS non sécurisés ou hostile. Plusieurs sécurités sont proposées par Sihag et al. [5], Haupert et al. [3] pour répondre à ce besoin :

- anti-falsification : vérification et protection de l'intégrité du code et des données de l'application afin d'éviter une modification du code par un attaquant
- anti-accrochage : vérification et protection de l'intégrité des appels de fonctions afin d'éviter l'interception d'un appel de fonction par un attaquant
- anti-débugueur : vérification de la présence de débugueur afin de complexifier la rétro-ingénierie
- anti-émulateur : vérification de l'environnement d'exécution afin de complexifier la rétro-ingénierie
- obfuscation : complexification du code dans le but de le rendre difficilement compréhensible afin de complexifier la rétro-ingénierie
- chiffrement en boite blanche : chiffrement du code ou des données de l'application afin de complexifier la rétro-ingénierie
- anti-root : vérification des droits possédés par l'utilisateur afin d'empêcher l'attaquant d'accéder aux données avec les droits de super-utilisateur
- anti-enregistreur de frappes : vérification de l'intégrité du clavier afin d'éviter le vol des données

- anti-lecteur d'écran : vérification et protection de l'affichage afin d'éviter le vol de données

- communications sécurisées : protection des données lors des communications réseau afin d'éviter l'interception des données

Sihag et al. [5] ainsi que Haupert et al. [3] proposent une dernière méthode de renforcement nommée liaison avec l'appareil. Cette méthode consiste à empêcher l'exécution d'une application à l'exception d'un ensemble autorisé d'appareil. Ces appareils sont identifiés à l'aide d'un identifiant physique unique. Dans le cas ou l'appareil n'est pas autorisé, l'application ne fonctionnera pas. Agrawal et al. [85] présentent une solution avec laquelle une application est chiffrée et ne se déchiffre afin d'être utilisée que si elle est exécutée sur l'appareil d'un utilisateur autorisé. Cette méthode se base sur l'utilisation d'identifiant physique contenue sur l'appareil de l'utilisateur. Bien que cette technique soit efficace, elle possède un usage restreint à des applications dont les utilisateurs souhaitent associer un téléphone unique.

Localisation de Vulnérabilités. Afin de pouvoir se protéger, il est nécessaire de savoir si une vulnérabilité affecte une application donnée. Les solutions suivantes sont des méthodes de détection des différentes vulnérabilités face auxquelles le blindage a pour objectif d'établir des protections.

Sihag et al. [5], Haupert et al. [3] mettent en avant la même méthode de détection de la falsification en réalisant un test d'intégrité à l'aide de certificats. Un certificat correspond à une application si le calcul de la somme de contrôle sur celle-ci produit le même résultat que celui donné par le certificat. Le calcul de la somme de contrôle permet, à partir des données du paquet de l'application, de calculer une donnée plus petite qui est celle donnée dans le certificat. La somme de contrôle donne un résultat identique pour un même bloc de données en entrée, mais a une très faible probabilité de produire un résultat identique pour deux blocs de données différents en entrée.

Pour la détection d'accrochage, Sihag et al. [5], Haupert et al. [3] proposent à nouveau la même méthode qui consiste à détecter des cadriciels d'accrochage les plus connus (ex. *Cydia Substrate*[86], *Frida*[87], *XPosed*[88]).

La détection de débugueur est dépendante du type de débogage utilisé. Sous *Android*, le débogage peut se faire de deux façons. La première est l'utilisation du débugueur *Java*. C'est celle qui est utilisée par les outils de développements *Android*. La seconde est l'utilisation de débugueur natif en *C/C++*. Cette méthode peut être mise en place manuellement par un développeur s'il a réalisé des implémentations en code natif ou afin de faire du débogage avancé. Pour le niveau *Java*, une détection triviale proposée par Sihag et al. [5], Haupert et al. [3] est l'utilisation d'une variable booléenne dans l'interface de programmation (API) *Android* permettant de savoir si un débugueur est attaché à l'application actuelle. Pour le niveau *C/C++*, Haupert et al. [3] se basent sur *ptrace*, la fonction de débogage intégré au *C/C++*, et sa caractéristique à ne pouvoir être mis en place qu'une seule fois en se connectant eux-mêmes à *ptrace*. Sihag et al. [5] vérifient les temps d'exécution afin de détecter des ralentissements dus au débugueur, ou en utilisant les signaux afin de voir si un débugueur les intercepte.

La détection d'émulateur se base sur les différences qu'il peut y avoir entre un environnement émulé et un vrai environnement. Sihag et al. [5] présentent une liste d'heuristiques. Elles peuvent être soit statiques à travers des valeurs par défaut dans l'environnement, soit dynamique avec des traces laissées par l'émulateur durant son exécution.

Des méthodes similaires sont utilisées par Sihag et al. [5], Haupert et al. [3] pour détecter la présence de root :

- La détection de paquet associé au root
- La vérification de la présence de fichier ou dossier n'existant que sur des appareils root
- La vérification de la présence de processus appartenant à l'utilisateur *sudo*

La détection d'enregistreur de frappes et de lecteur d'écran est complexe. Sihag et al. [5], Haupert et al. [3] proposent une même solution pour ces deux points qui est l'utilisation d'une liste blanche afin de répertorier les applications de confiance ayant ces utilités.

Remédiation. Parmi les différentes vulnérabilités et remédiations proposées par Sihag et al. [5] et Haupert et al. [3], on distingue trois cas majeurs. Le premier cas regroupe les vulnérabilités qu'il est possible de détecter, mais pour lequel aucune remédiation spécifique n'est associée. Généralement, une solution générique est employée afin d'arrêter l'application, modifier la mémoire utilisée par l'application afin d'y inscrire de fausses valeurs ou encore envoyer une alerte à un serveur. Le second cas est celui où la vulnérabilité est détectable et sa remédiation connue. Le dernier est plus rare, il s'agit de celui où la vulnérabilité est difficilement, voir impossible, à détecter, mais une remédiation est connue. Dans ce cas, on applique généralement la remédiation de façon préventive. C'est sur les deux derniers cas que les deux études précédentes se penchent.

Afin de faire face aux enregistreurs de frappes, Sihag et al. [5] ainsi que Haupert et al. [3] avancent une solution commune qui est la mise à disposition d'un clavier sécurisé pour les applications qui le souhaitent.

Pour la remédiation des lecteurs d'écran, Sihag et al. [5], Haupert et al. [3] utilisent tous deux la propriété *FLAG_SECURE* qui permet d'indiquer à l'API *Android* que les captures d'écran sont interdites sur une activité.

Afin de mettre en place des communications sécurisées, Sihag et al. [5], Haupert et al. [3] mettent en avant le remplacement du *HTTP* par le *HTTPS*. Sihag et al. [5] vont plus loin en proposant d'utiliser *TLS* pour les requêtes *DNS*, la vérification des certificats *TLS* ainsi que le chiffrage bout-à-bout des données transmises sur le réseau.

Dans le cadre de la protection par liaison avec l'appareil, Sihag et al. [5] ainsi que Haupert et al. [3] proposent l'utilisation de *IMEI* qui est un identifiant unique pour chaque appareil et qui est réputé robuste. Comme tous les identifiants physiques, il a le désavantage d'être accessible par toutes les applications de l'appareil.

2.4 Problèmes et Question de Recherche

Suite à l'étude des travaux connexes, il est possible d'établir certaines limites aux travaux sur le blindage d'applications *Android*.

Sihag et al. [5], Haupert et al. [3] abordent plusieurs vulnérabilités ainsi que leurs moyens de détection, mais le milieu de la sécurité évoluant vite, de nouvelles méthodes de contournement de celles-ci sont apparus. Il est donc nécessaire de mettre à jour les différentes méthodes de détection. De plus, des vulnérabilités sont manquantes dans la liste proposée[1]. Il est donc nécessaire de mettre en place la même méthode de détection/remédiation pour ces vulnérabilités.

En réponse à ces limites, nous pouvons nous poser la question suivante :

QR : "Comment automatiser le processus de blindage d'une application mobile Android en détectant ses vulnérabilités et en y remédiant ?"

[1] Ce stage a été réalisé en collaboration avec une entreprise dans le domaine de la sécurité logicielle.

3

Contribution

3.1 Introduction

La sécurité logicielle est la branche de la sécurité informatique centrée sur les logiciels et les différentes méthodes de protection.

Ce domaine se divise en plusieurs sous-domaines selon l'approche adoptée. L'approche peut varier selon le type d'application ciblée (bureau, dorsal, mobile ou web) et la temporalité de sa mise en application ("source", "empaqueté" ou "exécuté"). Notre étude se situe dans la branche des applications mobiles *Android* "empaqueté". Au sein de cette branche, plusieurs actions de sécurisation sont possibles. Parmi la détection de maliciels, la détection de vulnérabilités, la remédiation et les tests, cette étude aborde la question de la remédiation et plus précisément la remédiation préventive. La remédiation préventive est un sous-ensemble de la remédiation qui utilise des méthodes préventives afin d'anticiper certaines vulnérabilités potentielles. Sa mise en œuvre dans le cadre des applications mobiles est plus souvent nommé blindage.

Le blindage en est qu'à ses balbutiements et l'état de l'art actuel possède certaines limites. Les méthodes de détection des vulnérabilités considérées par le blindage ne sont pas à jour et certaines vulnérabilités potentielles ne sont pas encore prises en compte.

L'objectif de ce stage et sa contribution est donc de répondre à ces limites en étoffant l'ensemble des vulnérabilités, des méthodes de détection ainsi que des remédiations. Ces nouvelles approches seront ensuite vérifiées à travers l'implémentation d'un outil.

3.2 Sélection et Détection des Vulnérabilités

Les deux premiers objectifs de ce stage sont de proposer un ensemble de vulnérabilités ainsi que leurs modes de détection associés. Nous allons présenter ces deux concepts de paire. Nous avons choisi de répartir les vulnérabilités en trois catégories.

3.2.1 Vulnérabilités Contextuelles

Nous proposons tout d'abord des vulnérabilités "contextuelles", c'est-à-dire des vulnérabilités dépendantes de l'état de l'application :

- Bluetooth
- Géolocalisation GPS
- NFC

Ce sont les trois principaux protocoles de communication autre que le *GSM* disponible sur les appareils *Android* modernes. Ces protocoles sont sensibles à certaines attaques et, s'ils ne sont en cours d'utilisation par l'application, sont des vulnérabilités potentielles.

On retrouve l'attaque de l'homme du milieu commune aux trois protocoles. Elle permet à un attaquant de se placer en tant qu'intermédiaire dans la communication entre deux entités sans que celles-ci en aient connaissance. Un schéma de cette attaque est visible à la figure 3.1.

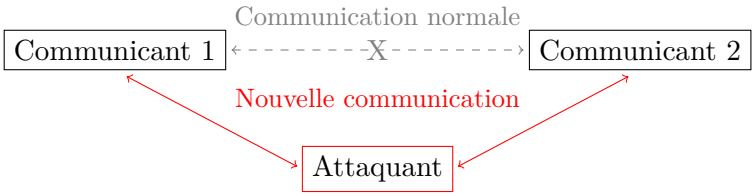


FIGURE 3.1 : Attaque de l'homme du milieu

De plus, la géolocalisation est sensible à l'usurpation par exemple et le Bluetooth peut être utilisé pour obtenir des données contenues sur l'appareil.

Ces vulnérabilités sont triviales à détecter puisque l'API *Android* propose des méthodes pour connaitre l'état des différentes connexions. La difficulté les concernant vient de leur aspect contextuel qui rend leur généralisation et leur protection automatique complexe.

3.2.2 Vulnérabilités Environnementales

Les vulnérabilités, désignées comme environnementales, sont des vulnérabilités dues à l'appareil sur lequel l'application s'exécute. Nous allons dans un premier temps présenter les vulnérabilités proposées par Haupert et al. [3] ainsi que Sihag et al. [5], pour lesquelles nous avons revu et mis à jour les méthodes de détection pour ensuite se concentrer la proposition de nouvelles vulnérabilités avec les détections associées.

3.2.2.1 Mise à jour des Détections des Vulnérabilités Établies

Accrochage. L'accrochage est une pratique permettant l'interception d'évènements et notamment d'appels de méthodes. Cette pratique peut être utilisée à des fins de sécurité dans le but de modifier dynamiquement un comportement vulnérable. Mais c'est plus généralement par des attaquants qu'elle est utilisée afin d'insérer un comportement malicieux une fois l'application en marche. Sa mise en place est complexe manuellement, car elle nécessite la modification du code binaire en mémoire afin de changer l'adresse

cible des appels de méthodes par une adresse d'un espace mémoire possédé par l'attaquant. Des cadriciels existent afin de faciliter cette tâche : *XPosed* par rovo89 [88], *Cydia Substrate* par SaurikIT [86], *Frida* par Ravnås [87]. La détection de l'accrochage est complexe, notamment s'il est réalisé manuellement, car elle nécessite un parcours et une compréhension de la mémoire. Une méthode de détection plus accessible est la détection des cadriciels connus. C'est cette méthode qui est utilisée par Haupert et al. [3], Sihag et al. [5]. Plusieurs méthodes réalisables en *Java* ou en *C/C++*, sont proposées par Haupert et al. [3], Sihag et al. [5], OWASP [89], Darvin [90], Bergman [91], Darvin [92], Mueller [93], Senyuuri [94]. Voici la liste des détections avec des exemples de valeurs suspectes :

- Détection de paquets connus[91]
  de.robv.android.xposed.installer, com.saurik.substrate
- Détection d'appels connus dans les traces d'exécution[91]
  de.robv.android.xposed.XposedBridge, com.saurik.substrate
- Détection de méthodes natives innatendues[91]
- Détection d'espaces mémoire chargés connus[91, 92]
  XposedBridge, com.saurik.substrate, re.frida, frida-agent, frida-gadget
- Détection de *fridaserver* dans les processus actifs[93]
- Vérification de l'écoute de *fridaserver* sur le protocole *D-Bus*[92]

Débugueur. Le débugueur *Android* permet, comme beaucoup de débugueurs, une exécution pas à pas, l'ajout de point d'arrêt ou encore l'inspection de la mémoire. Cette option facilite le développement en permettant d'analyser le programme pendant son exécution, mais aussi la rétro-ingénierie et le vol de données à un attaquant. La détection d'un débugueur peut se faire à l'aide d'une méthode renvoyant un booléen indiquant si un débugueur est connecté ou non, visible au listage 1. Cette technique ne fonctionne que si le débugueur est connecté au niveau du code *Java*.

```
Debug.isDebuggerConnected()
```

Listage 1: Détection de débugueur Java

Émulation. L'émulation permet de simuler un appareil physique de façon virtuelle. Les émulateurs *Android* sont très courants, notamment pendant la phase de développement d'application, afin de tester une diversité d'appareil possible. Cependant, l'émulateur peut permettre d'accéder à de nombreuses informations qu'un support physique ne permet à travers la lecture de la mémoire ou de l'inspection du code en exécution. Bien que pratique pour le développement, un attaquant peut se servir de ces capacités pour effectuer plus facilement de la rétro-ingénierie ou pour voler des données. La détection des émulateurs est une tâche précise puisqu'elle se base sur le fait de trouver des différences entre un appareil physique et un virtuel. Reproduire à l'identique un environnement réel

est compliqué et de subtiles différences peuvent se glisser dans les émulateurs. Voici une liste des points qui peuvent différer entre appareil physique et émulateur, ainsi que des exemples de valeurs incorrectes :

- Caractéristiques de Build par défaut (FINGERPRINT, MODEL, DEVICE, ...)
  generic, google_sdk, Emulator, Andy, nox, vbox86, goldfish, ...
- Présence de fichiers connus
  /dev/socket/genyd, fstab.nox, fstab.andy, fstab.vbox86, /dev/socket/qemud, ...
- Service téléphonique par défaut (Line1Number, NetworkOperatorName, ...)
  15555215554, 000000000000000, 15552175049, android, ...
- Présence de passerelles avec l'émulateur (/proc/tty/drivers, /proc/cpuinfo)
  goldfish
- Présence de paquets connus
  com.vphone, com.bignox, com.nox.mopen.app, me.haima, com.bluestacks, cn.itools, com.kop, com.kaopu, com.microvirt, com.bignox.app, com.google.android.launcher.layouts.genymotion

Utilisation d'un appareil "root". L'utilisation d'un appareil root indique que l'utilisateur de l'appareil *Android* a les droits d'administration. Par sécurité, ce sont des droits que les constructeurs ne laissent pas. Certains utilisateurs obtiennent ces droits afin d'apporter des modifications de personnalisation qui ne sont pas autorisées par défaut sur *Android*. Mais des attaquants utilisent aussi les droits d'administrateur afin d'accéder à des données sécurisées. *Android* utilise un principe de bac à sable, *Sandbox* en anglais, pour les applications afin de les isoler les unes des autres, mais l'obtention des droits d'administrateur permettent d'outrepasser ce principe et d'accéder à toutes les données. La détection du root se fait principalement à travers la présence de traces laissées par le processus. Cette vérification peut se faire en *Java* ou en *C/C++* pour renforcer la sécurité. Voici une compilation des différentes traces qui peuvent exister sur un appareil root avec des exemples de leurs valeurs sur des appareils root :

- Présence de fichier connus[95, 89]
  /system/app/Superuser.apk, /system/etc/init.d/99SuperSUDaemon, /dev/com.koushikdutta.superuser.daemon/, /system/xbin/daemonsu
- Présence de binaires administrateurs[95]
  /data/local/su, /data/local/bin/su, /data/local/xbin/su, /sbin/su, /su/bin/su, ...
- Présence de *su* dans le *PATH*[89]
- Possibilité d'exécuter la commande *su*[95, 89]
- Présence de *su* dans les processus[95, 89]
  supersu, superuser
- Présence de paquets connus[95, 89]
  com.thirdparty.superuser, eu.chainfire.supersu, com.noshufou.android.su, com.koushikdutta.superuser, com.zachspong.temprootremovejb, com.ramdroid., com.topjohnwu.magisk

- Accessibilité de fichiers systèmes en écriture[89]
  /bin, /data, etc, /proc, /sys, /system

- Valeur du *Build.TAGS* suspecte[95, 89]
  test-keys

- Présence de *Magisk* par Wu [96] dans les espaces mémoires chargés[90]
  magisk, core/mirror, core/img

3.2.2.2 Proposition de Vulnérabilités et des Détections Associées

Modification de la table de routage. Dans *Android*, il est possible à l'utilisateur d'ajouter des associations entre nom de domaine et adresse IP afin d'éviter la résolution du nom de domaine par DNS. Cette fonctionnalité peut permettre à un attaquant, dans le cas d'une application client-serveur, d'associer l'IP d'un serveur malveillant au nom de domaine utilisé par l'application. La détection de cette modification ne peut être fait qu'en demandant la résolution de domaine à *Android* et en comparant l'IP obtenue avec celle attendue, ce qui rend sa détection peu flexible.

Connexion à un Wifi public. Un Wifi public est un réseau qui ne demande pas d'identification et qui est généralement placé dans un lieu public ou de grande fréquentation. Bien qu'il offre un accès facile à internet à ses utilisateurs, il les place aussi dans un réseau commun. Un attaquant peut alors observer et intercepter le trafic des utilisateurs du réseau en y étant lui-même connecté. La détection d'un tel type de réseau peu se faire à l'aide de l'API *Android* qui propose des méthodes pour obtenir les caractéristiques des réseaux Wifi tel que présenté au listage 2.

```
// On récupère le réseau actuellement connecté
ConnectivityManager connectivity = (ConnectivityManager)
↪  context.getSystemService(Context.CONNECTIVITY_SERVICE);
NetworkCapabilities capabilities =
↪  connectivity.getNetworkCapabilities(connectivity.getActiveNetwork());

// On vérifie si le réseau actuel est un réseau Wifi
if(capabilities != null &&
↪  capabilities.hasTransport(NetworkCapabilities.TRANSPORT_WIFI))
{
    WifiInfo info = (WifiInfo) capabilities.getTransportInfo();
    return info.getCurrentSecurityType() == WifiInfo.SECURITY_TYPE_OPEN;// On teste le
    ↪  type de sécurité du Wifi
}
else
{
    return false;
}
```

Listage 2: Détection du Wifi public

Utilisation d'un VPN. Un *VPN* permet de se connecter à un réseau à distance à l'aide d'un tunnel chiffré. Cette technique qui était à l'origine faite pour se connecter à des réseaux privés et confidentiels à distances de façon sûre, a aujourd'hui été détournée de son utilisation première afin de tromper les sites web sur l'origine réelle des requêtes. Cette nouvelle utilisation s'est démocratisée auprès du grand public avec l'émergence de sociétés offrant des *VPN* à travers le monde permettant ainsi de choisir sa localisation et de cacher son identité. L'utilisation actuelle des *VPN* repose sur un tiers de confiance qui est le fournisseur de *VPN*. Mais rien ne prouve à l'application que ce tiers est réellement de confiance. La détection de l'utilisation d'un *VPN* peut se faire de deux façons. Sur les téléphones modernes, une option de connexion à un *VPN* est intégrée directement à l'OS. Dans ce cas de figure, *Android* propose des méthodes afin de savoir si cette option est active. Dans les autres cas, il est possible de supposer l'utilisation d'un *VPN* à l'aide de l'IP actuelle de l'appareil en le comparant à certains préfixes d'IP connu tel que montré au listage 3.

```
List<String> IP_PREFIXES = Arrays.asList(
        "10.", "172.16.", "172.17.", "172.18.", "172.19.",
        "172.20.", "172.21.", "172.22.", "172.23.", "172.24.",
        "172.25.", "172.26.", "172.27.", "172.28.", "172.29.",
        "172.30.", "172.31.", "192.168.", "127."
);

Enumeration<NetworkInterface> networks = NetworkInterface.getNetworkInterfaces();
while(networks != null && networks.hasMoreElements())
{
    NetworkInterface network = networks.nextElement();

    Enumeration<InetAddress> addresses = network.getInetAddresses();
    while(addresses != null && addresses.hasMoreElements())
    {
        InetAddress address = addresses.nextElement();

        for(String prefix : IP_PREFIXES)
        {
            // L'une des interfaces réseaux de l'appareil a une adresse IP commençant
            ↪  par l'un des préfixes
            if(Objects.requireNonNull(address.getHostAddress()).startsWith(prefix))
            {
                return true;
            }
        }
    }
}

return false;
```

Listage 3: Détection de *VPN*

Connexion à une antenne GSM pirate. La connexion à une antenne *GSM* pirate permet à un attaquant de réaliser une attaque de l'homme du milieu sur ce réseau. L'attaquant utilise alors une fausse antenne à laquelle l'appareil va se connecter automa-

tiquement en explorant les réseaux disponibles. Ce type d'attaque est difficile à détecter. Il existe cependant une heuristique se basant sur la taille de l'antenne. Ce genre d'antenne étant généralement plus petite que les traditionnelles, elles possèdent une portée et une intensité plus faible. On peut donc se baser sur l'intensité du signal reçu par le téléphone pour décider si l'appareil est connecté à une antenne pirate. Cette technique est loin d'être totalement fiable, notamment dans des zones ou la couverture réseau n'est pas particulièrement bonne. Pour mettre cette technique en place, l'API *Android* propose des méthodes pour obtenir le statut du réseau visible au listage 4.

```
TelephonyManager manager = (TelephonyManager)
↪  context.getSystemService(Context.TELEPHONY_SERVICE);
if(manager != null)
{
    // On considère ici, en exemple, qu'un signal de 0 ou 1 est un risque (Le niveau
    ↪  varie de 0 à 4 inclut où 0 est le signal de plus mauvaise qualité)
    return manager.getSignalStrength().getLevel() <= 1;
}
else
{
    return false;
}
```

Listage 4: Détection d'une antenne pirate

Utilisation du mode développeur. Le mode développeur est, comme son nom l'indique, un mode de fonctionnement donnant des options supplémentaires aux développeurs. Il permet d'accéder à des options telles que l'affichage d'information sur le rendu, d'activer le contrôle par USB ou encore d'autoriser le GPS virtuel. Ces options, activable avec une simple manipulation, permettent alors à un attaquant d'outrepasser certains fonctionnements de l'appareil. Le mode développeur étant une option interne à *Android*, elle est facile à détecter en vérifiant simplement l'état du paramètre à l'aide d'une méthode présenté au listage 5.

```
// La valeur numérique est en fait un booléen avec 0 pour false et 1 pour true
Settings.Global.getInt(context.getContentResolver(),
↪  Settings.Global.DEVELOPMENT_SETTINGS_ENABLED, 0) == 1
```

Listage 5: Détection du mode développeur

Utilisation d'un appareil dépassé. Tout comme beaucoup de gros projets, *Android* effectue des maintenances de sécurité sur un ensemble de versions plus anciennes que la version actuelle. Cette pratique permet de conserver les appareils pour un usage courant. Cependant, cette maintenance prend fin au bout d'un certain temps, lorsque les développeurs considèrent que la version est devenue obsolète. L'exécution d'une application sur un appareil considéré obsolète l'expose à des failles de sécurités importantes. La détection d'un appareil obsolète peut se faire en comparant la version de l'appareil avec

la dernière version en maintenance[1] à l'aide de constantes de l'API *Android* présentée au listage 6.

```
Build.VERSION.SDK_INT <= Build.VERSION_CODES.R
```

Listage 6: Détection d'une version dépassé d'Android

3.2.3 Vulnérabilités Statiques

Les vulnérabilités dites statiques sont les vulnérabilités qui ne dépendent que de l'application et de son fonctionnement :

- Falsification
- Lecteur d'écran
- Enregistreur de frappes
- Communications non sécurisées
- Absence d'obfuscation et de chiffrement

Ces vulnérabilités ne possèdent pas de détection à proprement parler, ce sont simplement des bonnes pratiques de sécurité à ajouter à une application.

Toutes ces vulnérabilités sont reprises de Haupert et al. [3] ainsi que de Sihag et al. [5]. Nous discuterons de modifications potentielles des remédiations associées dans la prochaine section. Une exception est faite ici concernant la liaison avec l'appareil qui a été proposée dans les articles cités. Cette pratique étant très spécifique à un type de besoin (notamment dans le milieu bancaire) et pas adapté au cas général, nous avons donc choisi de la mettre de côté. Si cette pratique avait dû être conservée, elle appartiendrait à cette famille.

3.3 Remédiation des Vulnérabilités

Afin d'ajouter de la granularité et du détail dans les remédiations des vulnérabilités, nous avons proposé plusieurs méthodes de remédiation que nous avons triées selon trois types.

3.3.1 Remédiation Générale

Le premier type proposé est celui de la remédiation générale. Il ne contient en fait qu'une seule méthode. C'est la méthode employée dans le cas où aucune réparation n'est possible pour une vulnérabilité donnée. Ce cas général consiste à arrêter l'exécution de l'application afin de protéger ses données et son intégrité. Pour cela, il s'agit d'essayer de tuer le processus courant et de faire appel à *exit()* qui permet d'arrêter le programme *Java* en cours. Ce morceau de code est présenté au listage 7.

[1] *Android 11* au moment de l'écriture (Wikipedia)

```
android.os.Process.killProcess(android.os.Process.myPid());
System.exit(1);
```

Listage 7: Code d'arrêt complet de l'application

3.3.2 Remédiations Ciblées

Le second type de remédiation proposé englobe les remédiations ciblées. Elles sont adaptées à une vulnérabilité précise afin de la protéger et de permettre à l'application de continuer son fonctionnement. On retrouve deux remédiations dans ce cas-là parmi les vulnérabilités décrites précédemment.

La première remédiation ciblée est celle liée aux communications *GSM*. La vulnérabilité associée est la connexion à une antenne *GSM* pirate qui ne rend vulnérable que l'utilisation de ce réseau. Il est alors possible d'encapsuler les communications avec celui-ci. Pour cela, il est dans un premier temps nécessaire de localiser les appels à l'interface *GSM* d'*Android*. Dans ce cas-ci, une unique méthode dans l'API permet de réaliser l'envoi de SMS visible au listage 8. Dans un second temps, il est nécessaire d'encapsuler l'appel à cette méthode dans une branche conditionnelle vérifiant si le réseau *GSM* est sûr. Dans le cas contraire, il est possible d'afficher un message *Toast* afin de prévenir l'utilisateur que cette action n'est pas réalisable. Un exemple du résultat souhaité après remédiation est donné en *Java* au listage 9 avec l'utilisation d'un *if* et au listage 10 avec un *throw* qui produisent des résultats similaires.

```
SmsManager manager = SmsManager.getDefault();
manager.sendTextMessage("0000000000", null, "Hello World !", null, null);
```

Listage 8: Méthode d'envoi de SMS

```
if(!isHackedAntennaAThreat(context))
{
    SmsManager manager = SmsManager.getDefault();
    manager.sendTextMessage("0000000000", null, "Hello World !", null, null);
}
else
{
    Toast.makeText(context, "GSM is not secure", Toast.LENGTH_SHORT).show();
}
```

Listage 9: Encapsulation d'un appel de méthode vulnérable avec un *if*

La seconde remédiation ciblée est similaire à la première, mais concerne le réseau internet. La démarche qui peut se réaliser en deux temps est la même. On commence avec la localisation des appels à l'API puis on termine avec l'encapsulation dans une branche conditionnelle. Elle diffère cependant dans la mise en œuvre de ces deux points.

Pour la localisation des appels à l'API, les méthodes réalisant des communications sur ce réseau sont nombreuses et d'autant plus si l'on prend en compte certaines librairies très largement utilisées. En effet, la méthode recommandée pour réaliser des appels HTTP a évolué au cours du développement d'*Android*. C'était dans un premier temps

```
try
{
    if(isHackedAntennaAThreat(context))
    {
        throw new ShieldingException("GSM is not secure");
    }

    SmsManager manager = SmsManager.getDefault();
    manager.sendTextMessage("0000000000", null, "Hello World !", null, null);
}
catch(ShieldingException exception)
{
    Toast.makeText(context, exception.getMessage(), Toast.LENGTH_SHORT).show();
}
```

Listage 10: Encapsulation d'un appel de méthode vulnérable avec un *throw*

l'utilisation d'une librairie *Apache* intégrée au kit de développement (SDK) qui a été recommandé pour ensuite évoluer vers les méthodes de la librairie standard de *Java* ainsi que des bibliothèques externes. Il est donc nécessaire de localiser non pas un mais tout un ensemble d'appels de méthodes. Voici une liste des différentes méthodes de requêtage associées à leur utilisation :

- Apache Http Client par Kalnichevski [97] :

  ```
  HttpClient httpclient = new DefaultHttpClient()
  ```

- Retrofit par Wharton [98] :

  ```
  Retrofit retrofit = new Retrofit.Builder().baseUrl("https://example.com/").build()
  ```

- OkHttp par Wilson [99] :

  ```
  Request request = new Request.Builder().url("https://example.com/").build()
  ```

- Volley par Davidson [100] :

  ```
  RequestQueue queue = Volley.newRequestQueue(this)
  ```

- HttpURLConnection :

  ```
  HttpURLConnection urlConnection = (HttpURLConnection) url.openConnection()
  ```

Pour l'encapsulation dans une branche conditionnelle, plusieurs vulnérabilités concernent le réseau internet. On retrouve l'utilisation d'un *VPN*, la connexion à un Wifi public et la modification de la table de routage. Il est donc nécessaire de tester l'ensemble de ces vulnérabilités avant de permettre la communication via internet.

3.3.3 Remédiations Statiques

Les remédiations statiques sont nommées comme telles car elles sont directement en lien avec les vulnérabilités statiques présentées précédemment. Ces remédiations sont des transformations de code dans le but de respecter certaines bonnes pratiques ou d'ajouter

une couche de protection. Dans un premier temps nous faire un rappel des remédiations proposées par Sihag et al. [5], Haupert et al. [3] pour lesquelles nous n'avons pas apporté aucun changement, puis nous discuterons de notre approche pour l'obfuscation et le chiffrement.

Remédiations Statiques Existantes. Afin de remédier à la falsification, il est nécessaire de vérifier l'intégrité de l'application par la vérification de la signature de l'APK.

La remédiation concernant un lecteur d'écran est la plus simple. Un indicateur est activable dans les activités *Android* afin que celle-ci soit protégée de tout enregistrement d'écran. Cet indicateur est nommé **FLAG_SECURE**

Pour remédier à un enregistreur de frappess, il est possible de mettre en place un clavier sûr dans le cadre du blindage et de l'associer à l'application afin que ce soit celui afficher durant son utilisation.

Afin de rendre les communications plus sécurisées, il est nécessaire de s'assurer que l'application utilise bien le protocole *HTTPS* au lieu de *HTTP* et de modifier les *URL*s qui ne sont pas en *HTTPS*. La pratique du *SSL Pinning* est aussi utilisable dans ce cas même si elle peut être déconseillée due aux besoins de maintenance par Developers [101]. En effet, cette méthode consiste à vérifier que le certificat renvoyé par une requête *HTTPS* (et donc TLS) correspondant à un certificat de confiance inclus dans l'application. Cependant, si le certificat est modifié par le serveur, alors il ne sera plus considéré comme étant de confiance, ce qui nécessitera une mise à jour logicielle pour rétablir la connexion. Le problème étant que certains utilisateurs ne réalisent pas régulièrement les mises à jour, ce qui peut leur faire conserver une version dysfonctionnelle. Enfin, une dernière solution possible est le chiffrement de bout en bout des données lors des communications. Cette technique ne peut être mise en œuvre que si les développeurs de l'application sont aussi ceux du serveur. En effet, il est nécessaire d'accorder les deux pour une communication chiffrée et, hormis le *HTTPS*, aucun standard n'est aujourd'hui communément mis en place sur les API publique.

Approche d'Ofuscation et Chiffrement Proposée. Concernant l'obfuscation et le chiffrement, la solution proposée est d'allier les deux. Deux composants sont à gérer indépendamment dans le cadre des applications *Android*, les ressources et le code. Concernant les ressources, il est possible de charger toutes les ressources dans une table associative liant "*id*" et ressource chiffrée. Il est ensuite nécessaire de remplacer les appels au système de ressource *Android* par des appels à la table des ressources qui déchiffre à la volée celle qui est demandée. Concernant le code, il est dans ce cas possible de chiffrer le fichier contenant le code au sein de l'APK et de le placer dans les *assets* de l'application. Il est ensuite nécessaire de remplacer ce fichier par un autre contenant une activité vide dont le travail est d'effectuer les vérifications des vulnérabilités listées précédemment avant de charger dynamiquement le fichier placé dans les *assets*. Cette double méthode permet de protéger complétement le code et les ressources des attaquants, mais a le désavantage de ralentir l'application selon sa taille à cause des calculs de déchiffrement.

3.4 SHIELD : Un outil pour une réalisation automatique du blindage d'applications mobiles Android

Les approches de détection et remédiations étant génériques et automatisables, nous avons choisi de les tester en les implémentant sous la forme d'un outil automatique de blindage. Nous avons développé l'outil**SHIELD** acronyme de Shielding and Hardening for Internal and Environmental Liabilities with Detection qui peut se traduire par Blindage et Renforcement des Risques Interne et Externe par Détection. L'objectif de cet outil est de mettre en place les détections de vulnérabilités et les remédiations associées telles que présentées précédemment sans l'intervention d'un développeur. Cet outil agit sur une application empaquetée prête au déploiement. Elle prend donc en entrée l'APK de l'application et doit produire un APK blindé.

3.4.1 Choix Techniques

Afin de réaliser cet outil, nous avons dû, dans un premier temps, choisir sous quelle forme nous allons manipuler l'APK. Étant un format de paquets compressé et compilé pour la partie code, il est difficile à lire tel quel. Plusieurs outils sont disponibles afin de transformer ce fichier en format plus facile à traiter. Une première méthode consiste à transformer l'APK en JAR, à l'aide de l'outil *dex2jar* par Pan [102]. Ceci permet ensuite d'utiliser tous les outils connus pour *Java*. Une seconde méthode est de décompresser l'APK et convertir le fichier *classes.dex* en une multitude de fichiers *Smali*. C'est ce que permet l'outil *smali/baksmali* par Gruver [103]. Notre choix s'est porté vers la seconde méthode. *Smali* a l'avantage d'être pensé pour la plateforme *Android* ainsi que d'être facilement compréhensible et modifiable. De plus, il n'y a pas de perte d'information entre le format du fichier *classes.dex* et celui du *Smali*.

Un outil existe afin de réaliser la démarche complète de décompression de l'APK, décompression des ressources et désassemblage avec *backsmali*, ainsi que la démarche inverse. Il se nomme *Apktool* par Tumbleson and Wisniewski [104].

Dans un second temps, il a fallu choisir le langage d'implémentation de l'outil. Ce choix a été dirigé par le besoin d'avoir un analyseur syntaxique pour le code *Smali* afin de construire un AST pour lire et modifier le code. Notre choix s'est porté vers le langage *Rust* qui a l'avantage d'offrir à la fois performance et fiabilité, et de proposer la bibliothèque recherchée développée par Whaley [105]. De plus, une bibliothèque de lecture de fichier manifeste android est aussi proposée par Rare [106], ce qui sera nécessaire pour le projet.

3.4.2 Démarche du Blindage

La démarche du blindage peut s'observer à deux échelles.

3.4.2.1 Échelle Macroscopique

À l'échelle macroscopique, on peut observer une démarche en trois temps. Tout d'abord, l'APK est désassemblé avec *Apktool*[104], puis blindé, pour être ensuite réassemblé à nouveau avec *Apktool*[104]. Cette échelle permet d'observer la séquence d'utilisation des différents outils. Elle est schématisée à la figure 3.2.

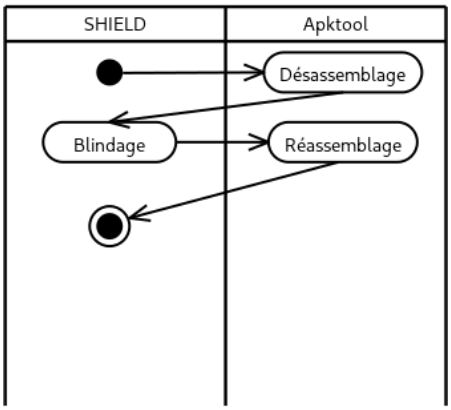


FIGURE 3.2 : Fonctionnement de SHIELD à l'Échelle Macroscopique

3.4.2.2 Échelle Microscopique

Par rapport à l'échelle microscopique, on peut observer l'ensemble des éléments liés à notre méthodologie de blindage. Celle-ci se décompose en quatre étapes. Elle peut être observé sur le schéma de la figure 3.3.

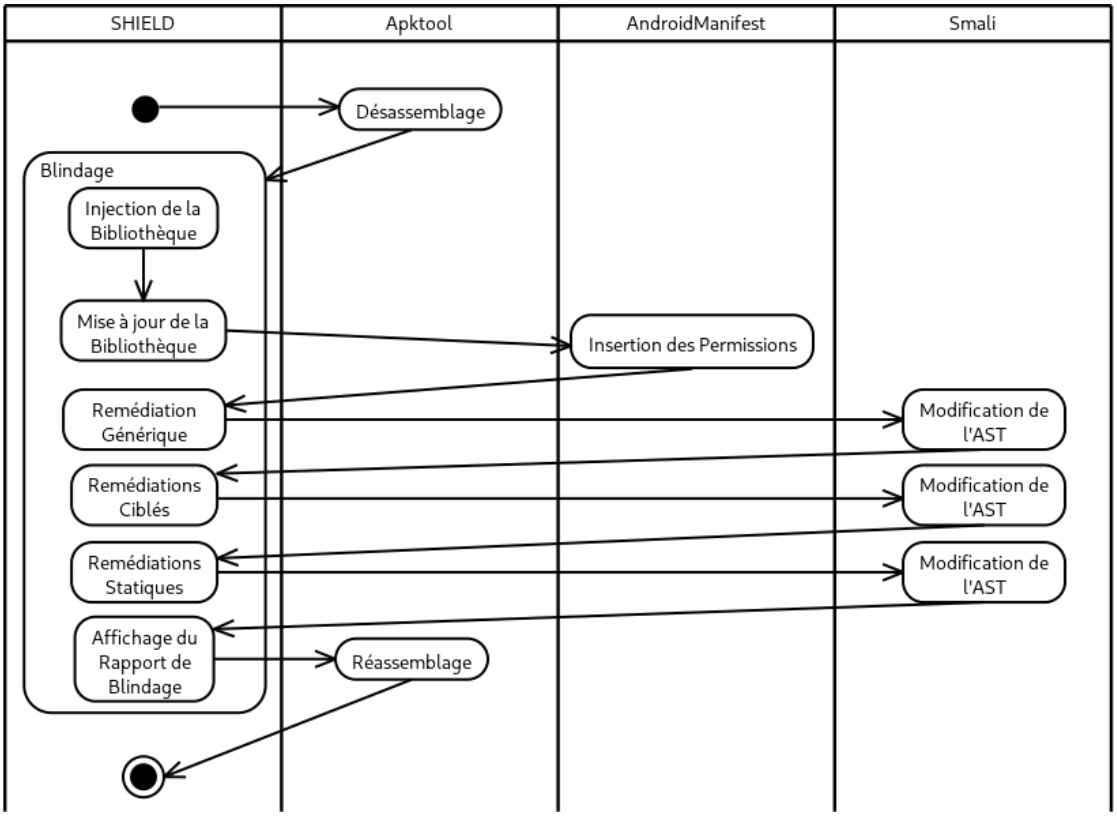


FIGURE 3.3 : Fonctionnement de SHIELD à l'Échelle Microscopique

La première est l'injection de la bibliothèque de blindage. En effet, afin de simplifier le développement, les méthodes de détection ainsi que plusieurs méthodes utilitaires ont été implémentées en *Java* puis exportées en *Smali*. Cette collection de méthodes a été nommée bibliothèque de blindage. Ce code est alors placé avec les sources *Smali* de l'APK par copie de fichier.

La seconde étape est la mise à jour de cette bibliothèque. Celle-ci possède plusieurs données qui doivent être indiquées de façon statique, car elles ne peuvent pas être connues à l'exécution. Ces informations sont le paquet *Java* racine de l'application ainsi que le nom de la classe *Application*.

La troisième étape est la mise à jour du manifeste android. Afin de pouvoir réaliser toutes les vérifications présentées précédemment, il est nécessaire de posséder certaines autorisations *Android*. Ces autorisations sont ajoutées à l'aide de la bibliothèque de lecture de fichier manifeste par Rare [106] au listage 11.

```
let mut manifest = android_manifest::from_reader(File::open(path)?)?;

for permission in vec![
    "android.permission.ACCESS_WIFI_STATE".to_string(),
    "android.permission.CHANGE_WIFI_STATE".to_string(),
    "android.permission.ACCESS_NETWORK_STATE".to_string(),
    "android.permission.BLUETOOTH".to_string(),
    "android.permission.BLUETOOTH_CONNECT".to_string(),
    "android.permission.POST_NOTIFICATIONS".to_string(),
    "android.permission.READ_PHONE_NUMBERS".to_string(),
    "android.permission.READ_PHONE_STATE".to_string(),
    "android.permission.QUERY_ALL_PACKAGES".to_string()
] {
    manifest.uses_permission.push(android_manifest::UsesPermission{
        name: Some(permission),
        max_sdk_version: None
    });
}

fs::write(path, android_manifest::to_string(&manifest)?)?;
```

Listage 11: Modification des permissions du Android Manifest

La dernière étape est la mise en place des remédiations décrites dans la précédente partie. Afin de clarifier le code de cette étape, la remédiation est réalisée en couche, où chaque couche travaille avec sa propre abstraction par rapport au code *Smali* et se base sur la couche du dessous. Une représentation visuelle de ces couches est faite à la figure 3.4.

Couche de Remédiation. La première couche est la plus abstraite, elle traite les remédiations à haut niveau en se concentrant sur une remédiation ciblée avec un objectif précis. On retrouve la fonction de remédiation *remedy_sms_request*, par exemple, qui va agir dans le cadre d'envoi de SMS sur un réseau vulnérable. Elle est visible au listage 12.

```
// Ajout de l'appel à la méthode utilitaire checkGSMOrThrow() de la bibliothèque de
↪  blindage avant chaque appel au SmsManager pour vérifier si le réseau \gls{gsm} est
↪  sécurisé ou lancer une exception le cas contraire
let patched_methods = manipulations::insert_before(source,
    |instruction| if let SmaliInstruction::Instruction(smali) = instruction {
        // Est la création d'un SmsManager

↪  smali.ends_with("Landroid/telephony/SmsManager;->getDefault()Landroid/telephony/SmsManager;")
    } else {
        false
    },
    "invoke-static {},
↪  Lfr/lirmm/shieldingproofofconcept/EnvironmentIntegrityChecker;->checkGSMOrThrow()V"
↪  // Appel à la bibliothèque de blindage
)?;

// Pour chaque méthode précédemment corriger, encapsulation de tout le corps de la
↪  méthode avec un try/catch afin de récupérer l'exception ShieldingException lancé et
↪  prévenir l'utilisateur
for (mut class, methods) in patched_methods {
    manipulations::wrap_whole_method_with_try_catch_in_class(
        &mut class,
        ShieldingMethodSignature::as_any_predicate(&methods), // Correspond à l'une des
↪  méthodes
        &vec!["Lfr/lirmm/shieldingproofofconcept/ShieldingException;"], // L'exception à
↪  capturer
        |method| {
            // Au moins une variable locale est nécessaire au catch
            if method.locals < 1 {
                method.locals = 1;
            }

            vec![
                SmaliInstruction::Instruction("const-string v0, \"SMS connection is not
↪  secure\"".to_owned()), // Message du Toast
                SmaliInstruction::Instruction("invoke-static {v0},
↪  Lfr/lirmm/shieldingproofofconcept/GlobalContext;->toastMessage(Ljava/lang/String;)V".to_owned()),
↪  // Affichage du Toast

                // Appel au return de la méthode
                SmaliInstruction::Instruction("const/4 v0, 0x0".to_owned()),

                if let TypeSignature::Void = method.signature.return_type {
                    SmaliInstruction::Instruction("return-void".to_owned())
                } else {
                    SmaliInstruction::Instruction("return v0".to_owned())
                }
            ]
        }
    )?;
```

Listage 12: Remédiation au requêtes SMS

Couche de Manipulation. La seconde couche est une collection de manipulations génériques. Les fonctions qui composent cette couche correspondent à des modifications minimales de l'AST. On y retrouve par exemple la manipulation *insert_before* qui permet d'insérer une instruction avant une autre. Elle est détaillée au listage 13.

```
pub fn insert_before<
    LP: Fn(&SmaliInstruction) -> bool
>(source: &Path, line_predicate: LP, to_insert: &str) -> Result<HashMap<SmaliClass,
↪   Vec<ShieldingMethodSignature>>, SmaliError> {
    let patched = patcher::patch(
        source,
        |_class| true,
        |_method| true,
        |_index, _len, instruction| line_predicate(instruction),
        0,
        |_method, _deleted| vec![SmaliInstruction::Instruction(to_insert.to_owned())]
    )?;

    Ok(patched)
}
```

Listage 13: Manipulation d'insertion avant une instruction dans l'AST

Couche de Modification. La troisième couche ne s'abstrait plus du tout de la structure de l'AST. C'est elle qui modifie directement cette structure de données. Elle est composée de trois fonctions permettant chacune de modifier l'AST à une échelle différente. On y retrouve *patch* qui permet d'appliquer un changement à l'ensemble du code, *patch_class* qui permet de modifier le code d'une classe ciblée, et *patch_method* qui permet de modifier le code d'une méthode précise. Le code de cette dernière est présenté au listage 14.

Dans les trois cas, la modification se fait de la même façon. Le code *Smali* étant une suite ordonnée d'instructions, c'est une technique de manipulation de tableau qui est utilisée ici. La méthode est celle du *splice*[2] que l'on peut retrouver dans certains langages ou bibliothèques.

Cette méthode nécessite trois paramètres, un index i à partir duquel il est nécessaire de réaliser les modifications, un nombre d'éléments n à supprimer et une liste d'élément L à insérer. Cette méthode supprime donc n éléments à partir de i puis y insère L.

Afin de la généraliser, l'utilisation d'index a été retirée puisqu'elle n'est pas la plus pertinente dans le cadre d'une suite d'instructions. L'index i a été modifié en un prédicat P qui, s'il est vrai, indique le début de la modification. La liste d'instructions L a été changée en une fonction G qui prend en paramètre les éléments supprimés et qui retourne les éléments à ajouter. Cette approche offre une contextualisation lors de l'ajout d'élément. C'est le code de cette méthode est montré au listage 14.

[2] https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/splice

```rust
fn patch_method<
    LP: Fn(usize, usize, &SmaliInstruction) -> bool,
    MU: Fn(&mut SmaliMethod, Vec<SmaliInstruction>) -> Vec<SmaliInstruction>
>(method: &mut SmaliMethod, line_predicate: LP, to_delete: usize, mutator: MU) ->
↪  Result<usize, SmaliError> {
    let mut locations: Vec<usize> = Vec::new(); // Index des lignes correspondant au
↪  line_predicate
    let mut offset = -0; // Décalage par rapport à l'index suite aux modifications

    // Recherche des index correspondat au line_predicate
    let instructions_len = method.instructions.len();
    for (index, instruction) in method.instructions.iter().enumerate() {
        if line_predicate(index, instructions_len, instruction) {
            locations.push(index);
        }
    }

    if line_predicate(instructions_len, instructions_len,
↪  &SmaliInstruction::Instruction(String::new())) {
        locations.push(instructions_len);
    }

    // Pour chaque ligne, on supprime les instructions puis on rajoute les nouvelles
    for location in &locations {
        let offsetted_index = (*location as isize + offset) as usize; // Index après
↪  recalcul suite aux modifications

        // Suppression des instructions
        let removed : Vec<SmaliInstruction> =
↪  method.instructions.drain(offsetted_index..(offsetted_index+to_delete)).collect();
        let removed_len = removed.len();

        // Génération des instructions à ajouter
        let created = mutator(method, removed);
        let created_count = created.len();

        // Ajout des nouvelles instructions
        method.instructions.splice(offsetted_index..offsetted_index, created);

        // Mise à jour de l'index suite aux modifications
        offset -= removed_len as isize;
        offset += created_count as isize;
    }

    Ok(locations.len())
}
```

Listage 14: Modification de l'AST d'une méthode

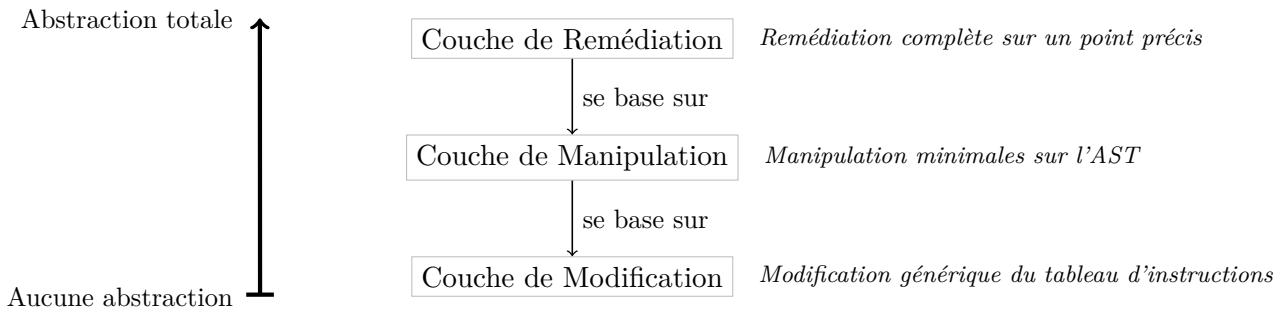


FIGURE 3.4 : Représentation en couche de la remédiation

3.5 Réponse à la Question de Recherche

En réponse à la question de recherche QR posée par rapport à ce travail de stage, nous avons proposé une solution permettant de détecter un ensemble de vulnérabilités, une collection de remédiations pour ces vulnérabilités ainsi qu'un outil permettant d'automatiser ce processus de blindage. Cet outil prend en entrée un APK et produit un APK blindé. Il se base sur une bibliothèque intégrant les différentes méthodes de détection définies ainsi que des méthodes utilitaires. L'outil s'inspire des techniques d'APR et des approches basées sur les patrons dans sa démarche de remédiation en retrouvant dans le code des instructions étiquetées comme vulnérables et en y insérant une protection.

4

Perspectives

Suite à ce stage, plusieurs perspectives s'ouvrent à différents niveaux.

Tout d'abord, des perspectives de poursuite du projet **SHIELD** sont envisageables. L'implémentation des remédiations statiques serait un angle de poursuite pertinent afin de compléter la prise en charge des remédiations proposées. L'ajout du support de *multidex* permettrait de grandement élargir le nombre d'applications sur lesquelles l'outil serait applicable. L'ajout de la détection et de la remédiation des débugueurs *C/C++* permettrait aussi une couverture plus grande de la détection des attaques. Enfin, l'implémentation totale des détections dans leurs versions *C/C++* afin de contourner certains accrochages possibles permettrait de réaliser des détections plus fiables.

D'autres perspectives se profilent d'un point de vue global dans le domaine du blindage. En effet, l'état de l'art tout comme ce stage présentent une mise en place du blindage aux applications *Android*. Ce système est effectivement l'un des plus présents sur le marché du mobile, mais il n'est pas le seul. Élargir l'étude du blindage aux autres systèmes tels que *iOS* permettrait un renforcement plus global de la sécurité des applications. De même que les attaquants cherchent sans cesse de nouveaux moyens de contourner les sécurités mises en place, il reste pertinent de poursuivre la recherche de nouvelles vulnérabilités à sécuriser et de nouveaux moyens de les détecter. Le cas s'est déjà présenté avec les cadriciels de accrochage et de root. Au fur et à mesure que des moyens de détecter les vulnérabilités sont trouvés, de nouvelles méthodes pour se camoufler et contourner ces détections apparaissent.

Enfin, le blindage est la mise en œuvre préventive de protection face à des vulnérabilités connues. Bien qu'il complexifie la tâche aux attaquants, avec les moyens nécessaires, il reste toujours possible de trouver de nouvelles failles. Il pourrait être intéressant de prendre du recul et de réfléchir à une approche d'exploration préventive des vulnérabilités qui n'ont pas encore été découvertes afin de trouver, avant les attaquants, les prochaines failles.

5

Conclusion

Durant ce stage, une étude de l'état de l'art de la sécurité logicielle et plus particulièrement de l'aspect blindage a été réalisée afin d'en voir les limites et d'en proposé une taxonomie. Deux aspects majeurs présentaient des limites dans l'état actuel des choses, les techniques de remédiations et l'automatisation du processus de blindage.

Nous avons alors proposé des approches de blindage automatisables, ainsi que l'outil **SHIELD** permettant de les valider sur des APK. Ces approches intègrent des détections pour un ensemble de vulnérabilité ainsi que les remédiations associées afin de les appliquer sur n'importe quelle application *Android*. L'ensemble des vulnérabilités a été compilé à partir à l'état de l'art. Cet ensemble a été étoffé et les vulnérabilités mise à jour pour le compléter. Des remédiations génériques et spécifiques ont été établis afin de définir clairement l'action à mener pour chacune des vulnérabilités. L'automatisation de cette démarche s'est inspiré de celle de l'APR et notamment des approches basées sur les patrons. Elle se déroule en deux temps, localisation puis remédiation. La localisation se fait par rapport aux instructions définies comme vulnérables, la remédiation se fait grâce à des patrons préétablis manuellement. Cet outil fait cependant face à plusieurs limitations.

Des perspectives sont tout de même à envisager, tel que l'implémentation des sécurités manquantes ou la résolution de la compatibilité avec le *multidex*. Le monde de la sécurité logicielle est un jeu permanent du chat et de la souris où les attaquants cherchent constamment à contourner les nouvelles sécurités mises en place. Il est donc toujours nécessaire de développer de nouvelles détections et de consolider les protections existantes. Enfin, ce mémoire se penche sur le cas précis de la protection des applications *Android* mais il peut être intéressant de développer une démarche similaire pour les autres plateformes mobiles telles que *iOS*.

Bibliographie

[1] Eric Fischer. Cybersecurity issues and challenges. In *Cybersecurity Issues and Challenges*. LIBRARY OF CONGRESS WASHINGTON DC, 2017.

[2] Jin-Hyuk Jung, Ju Young Kim, Hyeong-Chan Lee, and Jeong Hyun Yi. Repackaging attack on android banking applications and its countermeasures. *Wireless Personal Communications*, 73 :1421–1437, 2013.

[3] Vincent Haupert, Dominik Maier, Nicolas Schneider, Julian Kirsch, and Tilo Müller. Honey, i shrunk your app security : The state of android app hardening. In *Detection of Intrusions and Malware, and Vulnerability Assessment : 15th International Conference, DIMVA 2018, Saclay, France, June 28–29, 2018, Proceedings 15*, pages 69–91. Springer, 2018.

[4] Dionisio Zumerle. Shielding d'application mobile, 2023. URL `https://www.pradeo.com/fr-FR/application-mobile-shielding`.

[5] Vikas Sihag, Manu Vardhan, and Pradeep Singh. A survey of android application and malware hardening. *Computer Science Review*, 39 :100365, 2021.

[6] Richard A Kemmerer. Cybersecurity. In *25th International Conference on Software Engineering, 2003. Proceedings.*, pages 705–715. IEEE, 2003.

[7] Stephen Weis. Protecting data in-use from firmware and physical attacks. *Black Hat*, 2014.

[8] Annette Tetmeyer and Hossein Saiedian. Security threats and mitigating risk for usb devices. *IEEE Technology and Society Magazine*, 29(4) :44–49, 2010.

[9] Fadi Mohsen and Mohammed Shehab. Android keylogging threat. In *9th IEEE International Conference on Collaborative Computing : Networking, Applications and Worksharing*, pages 545–552. IEEE, 2013.

[10] Nasser Abouzakhar. Critical infrastructure cybersecurity : a review of recent threats and violations. *ECCWS*, 2013.

[11] Syed Wasif Abbas Hamdani, Haider Abbas, Abdul Rehman Janjua, Waleed Bin Shahid, Muhammad Faisal Amjad, Jahanzaib Malik, Malik Hamza Murtaza, Mohammed Atiquzzaman, and Abdul Waheed Khan. Cybersecurity standards in the context of operating system : Practical aspects, analysis, and comparisons. *ACM Computing Surveys (CSUR)*, 54(3) :1–36, 2021.

[12] Shyam Nandan Kumar. Review on network security and cryptography. *International Transaction of Electrical and Computer Engineers System*, 3(1) :1–11, 2015.

[13] Brendon Harris and Ray Hunt. Tcp/ip security threats and attack methods. *Computer communications*, 22(10) :885–897, 1999.

[14] Mohammed M Alani and Mohammed M Alani. Osi model. *Guide to OSI and TCP/IP Models*, pages 5–17, 2014.

[15] Zuoguang Wang, Limin Sun, and Hongsong Zhu. Defining social engineering in cybersecurity. *IEEE Access*, 8 :85094–85115, 2020.

[16] Paul Perrotin, Nicolas Belloir, Salah Sadou, David Hairion, and Antoine Beugnard. Hos-ml : Socio-technical system adl dedicated to human vulnerability identification. In *2022 26th International Conference on Engineering of Complex Computer Systems (ICECCS)*, pages 1–6. IEEE, 2022.

[17] Paul Perrotin, Nicolas Belloir, Salah Sadou, David Hairion, and Antoine Beugnard. Using the architecture of socio-technical system to analyse its vulnerability. In *2022 17th Annual System of Systems Engineering Conference (SOSE)*, pages 361–366. IEEE, 2022.

[18] Ahmed Aleroud and Lina Zhou. Phishing environments, techniques, and countermeasures : A survey. *Computers & Security*, 68 :160–196, 2017.

[19] Edward Amoroso. Recent progress in software security. *IEEE Software*, 35(2) : 11–13, 2018.

[20] A Babate, M Musa, A Kida, and M Saidu. State of cyber security : emerging threats landscape. *International Journal of Advanced Research in Computer Science & Technology*, 3(1) :113–119, 2015.

[21] Lisandro Delia, Nicolas Galdamez, Pablo Thomas, Leonardo Corbalan, and Patricia Pesado. Multi-platform mobile application development analysis. In *2015 IEEE 9th International Conference on Research Challenges in Information Science (RCIS)*, pages 181–186. IEEE, 2015.

[22] Google. Flutter : Build apps for any screen, 2017. URL `https://flutter.dev/`.

[23] Microsoft. Xamarin : An app platform for building android and ios apps with .net and c#, 2023. URL `https://dotnet.microsoft.com/en-us/apps/xamarin`.

[24] Ecma International. Ecma-262 : Ecmascript® 2022 language specification, 2022. URL `https://www.ecma-international.org/publications-and-standards/standards/ecma-262/`.

[25] Yu Beng Leau, Wooi Khong Loo, Wai Yip Tham, and Soo Fun Tan. Software development life cycle agile vs traditional approaches. In *International Conference on Information and Network Technology*, volume 37, pages 162–167, 2012.

[26] Alessandro Mantovani, Luca Compagna, Yan Shoshitaishvili, and Davide Balzarotti. The convergence of source code and binary vulnerability discovery–a case study. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, pages 602–615, 2022.

[27] Mark Graff and Kenneth R Van Wyk. *Secure coding : principles and practices.* " O'Reilly Media, Inc.", 2003.

[28] Aakiel Abernathy, Xiaohong Yuan, Edward Hill, Jinsheng Xu, Kelvin Bryant, and Kenneth Williams. Sach : a tool for assisting secure android application development. In *SoutheastCon 2017*, pages 1–4. IEEE, 2017.

[29] Mukesh Kumar Gupta, Mahesh Chandra Govil, Girdhari Singh, and Priya Sharma. Xssdm : Towards detection and mitigation of cross-site scripting vulnerabilities in web applications. In *2015 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, pages 2010–2015. IEEE, 2015.

[30] Muchammad Naseer, Jack Febrian Rusdi, Nuruddeen Musa Shanono, Sazilah Salam, Zulkiflee Bin Muslim, Nur Azman Abu, and Iwan Abadi. Malware detection : Issues and challenges. In *Journal of Physics : Conference Series*, volume 1807, page 012011. IOP Publishing, 2021.

[31] Abhishek Kumar Pandey, Ashutosh Kumar Tripathi, Gayatri Kapil, Virendra Singh, Mohd Waris Khan, Alka Agrawal, Rajeev Kumar, and Raees Ahmad Khan. Trends in malware attacks : Identification and mitigation strategies. In *Critical Concepts, Standards, and Techniques in Cyber Forensics*, pages 47–60. IGI Global, 2020.

[32] Abdullah Qasem, Paria Shirani, Mourad Debbabi, Lingyu Wang, Bernard Lebel, and Basile L Agba. Automatic vulnerability detection in embedded devices and firmware : Survey and layered taxonomies. *ACM Computing Surveys (CSUR)*, 54 (2) :1–42, 2021.

[33] Qazi Mamoon Ashraf and Mohamed Hadi Habaebi. Autonomic schemes for threat mitigation in internet of things. *Journal of Network and Computer Applications*, 49 :112–127, 2015.

[34] Petar Čisar and Sanja Maravić Čisar. The framework of runtime application self-protection technology. In *2016 IEEE 17th International Symposium on Computational Intelligence and Informatics (CINTI)*, pages 000081–000086. IEEE, 2016.

[35] Bruce Potter and Gary McGraw. Software security testing. *IEEE Security & Privacy*, 2(5) :81–85, 2004.

[36] Mohd Ehmer Khan and Farmeena Khan. A comparative study of white box, black box and grey box testing techniques. *International Journal of Advanced Computer Science and Applications*, 3(6), 2012.

[37] Kai Huang, Zhengzi Xu, Su Yang, Hongyu Sun, Xuejun Li, Zheng Yan, and Yuqing Zhang. A survey on automated program repair techniques. *arXiv preprint arXiv :2303.18184*, 2023.

[38] W Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. A survey on software fault localization. *IEEE Transactions on Software Engineering*, 42(8) : 707–740, 2016.

[39] Kui Liu, Li Li, Anil Koyuncu, Dongsun Kim, Zhe Liu, Jacques Klein, and Tegawendé F Bissyandé. A critical review on the evaluation of automated program repair systems. *Journal of Systems and Software*, 171 :110817, 2021.

[40] Mark Weiser. Program slicing. *IEEE Transactions on software engineering*, pages 352–357, 1984.

[41] R Lyle. Automatic program bug location by program slicing. In *Proceedings 2nd international conference on computers and applications*, pages 877–883, 1987.

[42] Hiralal Agrawal, Richard A DeMillo, and Eugene H Spafford. Debugging with dynamic slicing and backtracking. *Software : Practice and Experience*, 23(6) : 589–616, 1993.

[43] Xiangyu Zhang, Haifeng He, Neelam Gupta, and Rajiv Gupta. Experimental evaluation of using dynamic slices for fault location. In *Proceedings of the sixth international symposium on Automated analysis-driven debugging*, pages 33–42, 2005.

[44] Hiralal Agrawal, Joseph R Horgan, Saul London, and W Eric Wong. Fault localization using execution slices and dataflow tests. In *Proceedings of Sixth International Symposium on Software Reliability Engineering. ISSRE'95*, pages 143–151. IEEE, 1995.

[45] Hiralal Agrawal, Richard A. De Millo, and Eugene H. Spafford. An execution-backtracking approach to debugging. *IEEE Software*, 8(3) :21–26, 1991.

[46] Manos Renieres and Steven P Reiss. Fault localization with nearest neighbor queries. In *18th IEEE International Conference on Automated Software Engineering, 2003. Proceedings.*, pages 30–39. IEEE, 2003.

[47] James A Jones and Mary Jean Harrold. Empirical evaluation of the tarantula automatic fault-localization technique. In *Proceedings of the 20th IEEE/ACM international Conference on Automated software engineering*, pages 273–282, 2005.

[48] Ben Liblit, Mayur Naik, Alice X Zheng, Alex Aiken, and Michael I Jordan. Scalable statistical bug isolation. *Acm Sigplan Notices*, 40(6) :15–26, 2005.

[49] Chao Liu, Long Fei, Xifeng Yan, Jiawei Han, and Samuel P Midkiff. Statistical debugging : A hypothesis testing-based approach. *IEEE Transactions on software engineering*, 32(10) :831–848, 2006.

[50] Eric Wong, Tingting Wei, Yu Qi, and Lei Zhao. A crosstab-based statistical method for effective fault localization. In *2008 1st international conference on software testing, verification, and validation*, pages 42–51. IEEE, 2008.

[51] Thomas Zimmermann and Andreas Zeller. Visualizing memory graphs. In *Software Visualization : International Seminar Dagstuhl Castle, Germany, May 20–25, 2001 Revised Papers*, pages 191–204. Springer, 2002.

[52] Xiangyu Zhang, Neelam Gupta, and Rajiv Gupta. Locating faults through automated predicate switching. In *Proceedings of the 28th international conference on Software engineering*, pages 272–281, 2006.

[53] Tao Wang and Abhik Roychoudhury. Automated path generation for software fault localization. In *Proceedings of the 20th IEEE/ACM international Conference on Automated software engineering*, pages 347–351, 2005.

[54] W Eric Wong and Yu Qi. Bp neural network-based effective fault localization. *International Journal of Software Engineering and Knowledge Engineering*, 19(04) : 573–597, 2009.

[55] W Eric Wong, Yan Shi, Yu Qi, and Richard Golden. Using an rbf neural network to locate program bugs. In *2008 19th International Symposium on Software Reliability Engineering (ISSRE)*, pages 27–36. IEEE, 2008.

[56] Lionel C Briand, Yvan Labiche, and Xuetao Liu. Using machine learning to support debugging with tarantula. In *The 18th IEEE International Symposium on Software Reliability (ISSRE'07)*, pages 137–146. IEEE, 2007.

[57] Richard A DeMillo, Hsin Pan, and Eugene H Spafford. Failure and fault analysis for software debugging. In *Proceedings Twenty-First Annual International Computer Software and Applications Conference (COMPSAC'97)*, pages 515–521. IEEE, 1997.

[58] Franz Wotawa, Markus Stumptner, and Wolfgang Mayer. Model-based debugging or how to diagnose programs automatically. In *Developments in Applied Artificial Intelligence : 15th International Conference on Industrial and Engineering Applications of Artificial Intelligence and Expert Systems IEA/AIE 2002 Cairns, Australia, June 17–20, 2002 Proceedings*, pages 746–757. Springer, 2002.

[59] S Nessa, M Abedin, W Eric Wong, L Khan, and Y Qi. Fault localization using n-gram analysis. In *Proceedings of the 3rd International Conference on Wireless Algorithms, Systems, and Applications*, pages 548–559, 2009.

[60] Peggy Cellier, Mireille Ducassé, Sébastien Ferré, and Olivier Ridoux. Formal concept analysis enhances fault localization in software. In *Formal Concept Analysis : 6th International Conference, ICFCA 2008, Montreal, Canada, February 25-28, 2008. Proceedings 6*, pages 273–288. Springer, 2008.

[61] Fan Long and Martin Rinard. Staged program repair with condition synthesis. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, pages 166–178, 2015.

[62] Barbara Jobstmann, Andreas Griesmayer, and Roderick Bloem. Program repair as a game. In *CAV*, volume 5, pages 226–238. Springer, 2005.

[63] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. Genprog : A generic method for automatic software repair. *Ieee transactions on software engineering*, 38(1) :54–72, 2011.

[64] Tao Ji, Liqian Chen, Xiaoguang Mao, and Xin Yi. Automated program repair by using similar code containing fix ingredients. In *2016 IEEE 40th Annual Computer Software and Applications Conference (COMPSAC)*, volume 1, pages 197–202. IEEE, 2016.

[65] Yi Wei, Yu Pei, Carlo A Furia, Lucas S Silva, Stefan Buchholz, Bertrand Meyer, and Andreas Zeller. Automated fixing of programs with contracts. In *Proceedings of the 19th international symposium on Software testing and analysis*, pages 61–72, 2010.

[66] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. Semfix : Program repair via semantic analysis. In *2013 35th International Conference on Software Engineering (ICSE)*, pages 772–781. IEEE, 2013.

[67] Xuan-Bach D Le, Quang Loc Le, David Lo, and Claire Le Goues. Enhancing automated program repair with deductive verification. In *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 428–432. IEEE, 2016.

[68] Rijnard Van Tonder and Claire Le Goues. Static automated program repair for heap properties. In *Proceedings of the 40th International Conference on Software Engineering*, pages 151–162, 2018.

[69] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. An empirical study on learning bug-fixing patches in the wild via neural machine translation. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 28(4) :1–29, 2019.

[70] Martin White, Michele Tufano, Matias Martinez, Martin Monperrus, and Denys Poshyvanyk. Sorting and transforming program repair ingredients via deep learning code similarities. In *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 479–490. IEEE, 2019.

[71] Yi Li, Shaohua Wang, and Tien N Nguyen. Dlfix : Context-based code transformation learning for automated program repair. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, pages 602–614, 2020.

[72] He Ye, Matias Martinez, and Martin Monperrus. Neural program repair with execution-based backpropagation. In *Proceedings of the 44th International Conference on Software Engineering*, pages 1506–1518, 2022.

[73] Chunqiu Steven Xia and Lingming Zhang. Less training, more repairing please : revisiting automated program repair via zero-shot learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 959–971, 2022.

[74] Rahul Gupta, Soham Pal, Aditya Kanade, and Shirish Shevade. Deepfix : Fixing common c language errors by deep learning. In *Proceedings of the aaai conference on artificial intelligence*, volume 31, 2017.

[75] Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. Automatic patch generation learned from human-written patches. In *2013 35th International Conference on Software Engineering (ICSE)*, pages 802–811. IEEE, 2013.

[76] Shin Hwei Tan and Abhik Roychoudhury. relifix : Automated repair of software regressions. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, volume 1, pages 471–482. IEEE, 2015.

[77] Xuan Bach D Le, David Lo, and Claire Le Goues. History driven program repair. In *2016 IEEE 23rd international conference on software analysis, evolution, and reengineering (SANER)*, volume 1, pages 213–224. IEEE, 2016.

[78] Kui Liu, Dongsun Kim, Tegawendé F Bissyandé, Shin Yoo, and Yves Le Traon. Mining fix patterns for findbugs violations. *IEEE Transactions on Software Engineering*, 47(1) :165–188, 2018.

[79] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F Bissyandé. Avatar : Fixing semantic bugs with fix patterns of static analysis violations. In *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 1–12. IEEE, 2019.

[80] Anil Koyuncu, Kui Liu, Tegawendé F Bissyandé, Dongsun Kim, Martin Monperrus, Jacques Klein, and Yves Le Traon. ifixr : Bug report driven program repair. In *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, pages 314–325, 2019.

[81] Lily Hay Newman. Hacker lexicon : What is application shielding ?, 2019. URL `https://www.wired.com/story/what-is-application-shielding/`.

[82] Parvez Faruki, Hossein Fereidooni, Vijay Laxmi, Mauro Conti, and Manoj Gaur. Android code protection via obfuscation techniques : past, present and future directions. *arXiv preprint arXiv :1611.10231*, 2016.

[83] Azzam Mourad, Marc-André Laverdiere, and Mourad Debbabi. Security hardening of open source software. *ResearchGate*, 2006.

[84] Promon. Promon shield, 2006. URL `https://promon.co/`.

[85] Shashank Agrawal, Estuardo Alpírez Bock, Yilei Chen, and Gaven Watson. White-box cryptography with global device binding from message-recoverable signatures and token-based obfuscation. In *Constructive Side-Channel Analysis and Secure Design : 14th International Workshop, COSADE 2023, Munich, Germany, April 3–4, 2023, Proceedings*, pages 241–261. Springer, 2023.

[86] SaurikIT. Cydia substrate : The powerful code modification platform behind cydia, 2012. URL `http://www.cydiasubstrate.com/`.

[87] Ole André Vadla Ravnås. Frida : Dynamic instrumentation toolkit for developers, reverse-engineers, and security researchers, 2013. URL `https://frida.re/`.

[88] rovo89. Xposed module repository, 2013. URL `https://web.archive.org/web/20220831060343/https://repo.xposed.info/`.

[89] OWASP. Android anti-reversing defenses, 2023. URL `https://mobile-security.gitbook.io/mobile-security-testing-guide/android-testing-guide/0x05j-testing-resiliency-against-reverse-engineering`.

[90] Darvin. Detecting magisk hide, 2019. URL `https://darvincitech.wordpress.com/2019/11/04/detecting-magisk-hide/`.

[91] Neil Bergman. Android anti-hooking techniques in java, 2015. URL `https://d3adend.org/blog/posts/android-anti-hooking-techniques-in-java/`.

[92] Darvin. Detect frida for android, 2019. URL `https://darvincitech.wordpress.com/2019/12/23/detect-frida-for-android/`.

[93] Bernhard Mueller. The jiu-jitsu of detecting frida, 2017. URL `https://web.archive.org/web/20170503155916/http://www.vantagepoint.sg/blog/90-the-jiu-jitsu-of-detecting-frida`.

[94] Senyuuri. Android pms hook detection, 2020. URL `https://github.com/senyuuri/pms-hook-detection`.

[95] Mat Joseph. Root inspector, 2014. URL `https://github.com/devadvance/rootinspector`.

[96] John Wu. Magisk : The magic mask for android, 2013. URL `https://github.com/topjohnwu/Magisk`.

[97] Oleg Kalnichevski. Httpclient, 2005. URL `https://hc.apache.org/httpcomponents-client-4.5.x/index.html`.

[98] Jake Wharton. Retrofit : A type-safe http client for android and java, 2010. URL `https://square.github.io/retrofit/`.

[99] Jesse Wilson. Okhttp : Http is the way modern applications network, 2011. URL `https://square.github.io/okhttp/`.

[100] Jeff Davidson. Volley : An http library that makes networking for android apps easier and most importantly, faster, 2012. URL `https://google.github.io/volley/`.

[101] Android Developers. Security with network protocols, 2022. URL `https://developer.android.com/training/articles/security-ssl`.

[102] Bob Pan. dex2jar : Tools to work with android .dex and java .class files, 2010. URL `https://github.com/pxb1988/dex2jar/wiki`.

[103] Ben Gruver. Smali/baksmali : An assembler/disassembler for the dex format used by dalvik, android's java vm implementation, 2009. URL `https://github.com/JesusFreke/smali/wiki`.

[104] Connor Tumbleson and Ryszard Wisniewski. Apktool : A tool for reverse engineering android apk files, 2016. URL `https://ibotpeaches.github.io/Apktool/`.

[105] Andrew Whaley. Smali crate : A pure rust implementation of a parser, writer and set of types for the smali file format, 2022. URL `https://crates.io/crates/smali`.

[106] Dodo Rare. Androidmanifest crate : An androidmanifest serializer and deserializer for rust, 2021. URL `https://crates.io/crates/android-manifest`.

Annexes

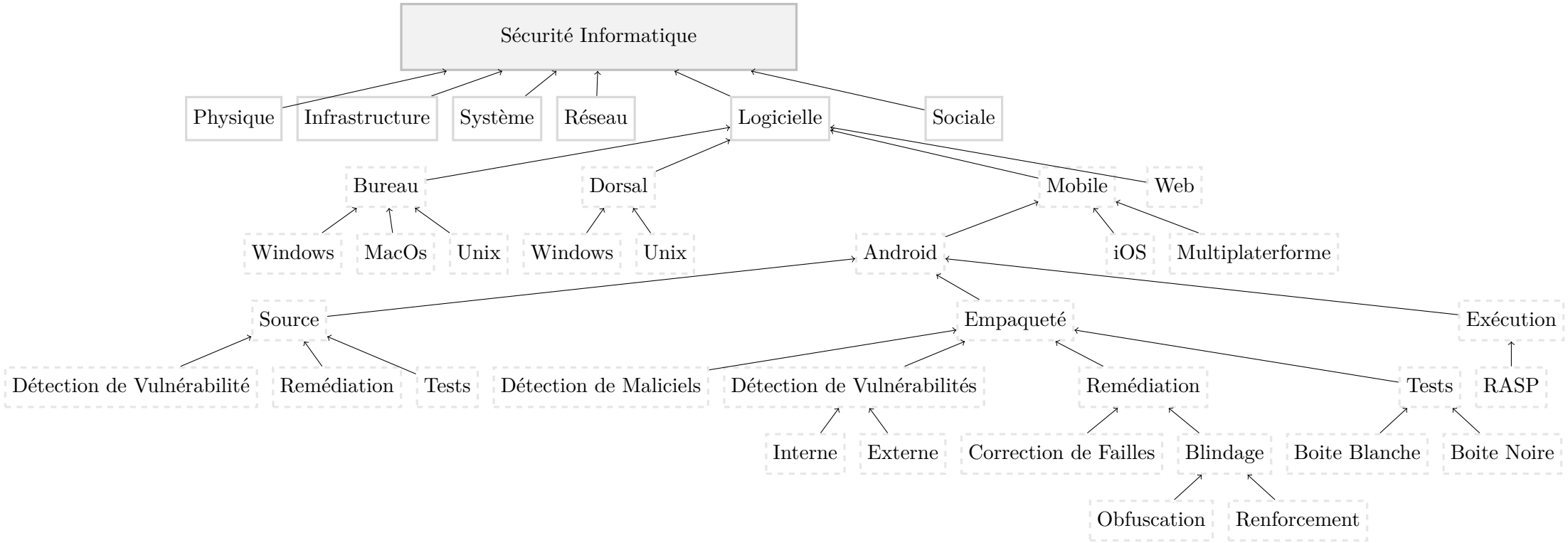


FIGURE 1 : Taxonomie du Domaine de Recherche